



Universitatea
Transilvania
din Brașov
FACULTATEA DE MATEMATICĂ
ȘI INFORMATICĂ

Programul de studii:
Aaaaaaa Aaaaa Aaaaaaa

Lucrare de licență LifeAlertPlus -- sistem IoT inteligent pentru monitorizarea și siguranța persoanelor vârstnice

Autor:

Stancu Nicol-Alexia

Coordonator științific:

Lect. Dr. Vasilescu Anca

Brașov, 2026



Cuprins

Listă de figuri	5
Listă de tabele	7
Listă de abrevieri	8
1 Introducere	10
1.1 Context actual	10
1.2 Motivația alegerii temei	10
1.3 Scopul lucrării	10
1.4 Obiective	11
1.5 Contribuții personale	11
1.6 Structura lucrării	12
2 Fundamente teoretice și tehnologii	13
2.1 Internet of Things	13
2.2 Senzori și dispozitive	14
2.2.1 Microcontrollerul ESP32-C3 Mini	14
2.2.2 Senzorul MAX30100 pentru puls și saturație de oxigen	14
2.2.3 Senzorul DS18B20 pentru temperatura corporală	14
2.2.4 Senzorul MPU6050 pentru mișcare și accelerație	15
2.2.5 Modulul GPS NEO-6M	15
2.2.6 Multiplexorul I2C TCA9548A	15
2.2.7 Sistemul de alimentare	15
2.2.8 Interfața utilizator pe dispozitiv	16
2.3 Prelucrarea datelor și inteligență artificială	16
2.3.1 Învățarea automată și problema de clasificare	16
2.3.2 Datele de antrenare și pregătirea lor	17
2.3.3 Sistemul hibrid de decizie	17
2.3.4 Explicabilitatea și evaluarea modelului	18
2.3.5 Clasificarea activității și arhitectura de tip microserviciu	18
2.4 Sisteme de alertare	19
2.4.1 Fiabilitate și problema alertelor false	19
2.4.2 Alertarea multi-canal	19
2.5 Tehnologii utilizate (backend, frontend, Docker etc.)	20
2.5.1 Backend-ul aplicației	20
2.5.2 Securitatea	21
2.5.3 Comunicarea în timp real	21
2.5.4 Frontend-ul aplicației	21
2.5.5 Containerizarea și implementarea	21
3 Analiza soluțiilor existente	22
3.1 Prezentarea soluțiilor actuale	22
3.1.1 Fitbit Sense 2 / Google Pixel Watch	22
3.1.2 Apple Watch (Series 9 / Ultra 2)	22
3.1.3 Philips Lifeline / Medical Guardian	22
3.1.4 Withings ScanWatch	23

3.1.5	Soluții academice și de cercetare	23
3.2	Limitări identificate	23
3.2.1	Inadecvarea pentru utilizatorii vârstnici	23
3.2.2	Lipsa monitorizării continue și a alertării inteligente	23
3.2.3	Platforme închise și lipsa personalizării	23
3.2.4	Costul și dependența de abonamente	23
3.2.5	Limitări ale soluțiilor academice	24
3.3	Poziționarea soluției propuse	24
4	Proiectarea sistemului	26
4.1	Arhitectura generală	26
4.1.1	Separarea în straturi a aplicației software	26
4.1.2	Microserviciul de inteligență artificială	26
4.1.3	Comunicarea între componente	27
4.1.4	Modelul de deployment	27
4.2	Componentele sistemului	27
4.2.1	Dispozitivul portabil	27
4.2.2	Backend-ul	28
4.2.3	Microserviciul de inteligență artificială	28
4.2.4	Aplicația web	28
4.2.5	Baza de date	28
4.2.6	Serviciile externe	28
4.3	Modelul de date	29
4.3.1	Relațiile dintre entități	30
4.3.2	Ștergerea datelor	30
4.3.3	Datele agregate și istoricul	30
4.4	Fluxuri de funcționare	31
4.4.1	Achiziția și transmiterea unei măsurători	31
4.4.2	Evaluarea stării prin modulul AI	31
4.4.3	Calculul health score-ului	31
4.4.4	Predicția de tendințe	33
4.4.5	Declanșarea și escaladarea unei alerte	34
4.4.6	Butonul de panică	35
4.4.7	Detecția căderilor pe dispozitiv	35
4.4.8	Profilul de activitate și detecția anomaliilor comportamentale	36
4.4.9	Configurarea rețelelor WiFi de la distanță	37
4.4.10	Feedback pentru alarme false	37
4.4.11	Politica de retenție a datelor	38
4.4.12	Sincronizarea datelor stocate local	38
4.4.13	Înrolarea utilizatorilor și asocierea dispozitivului	38
4.4.14	Invitarea medicului și exportul datelor	38
5	Implementarea aplicației	39
5.1	Hardware-ul dispozitivului portabil	39
5.1.1	Schema de conexiuni și maparea pinilor GPIO	39
5.1.2	Magistrala I ² C și multiplexarea TCA9548A	40
5.1.3	Sistemul de alimentare	40
5.1.4	Interfața fizică: LED-uri și butoane	41

5.1.5	Diagnoza la pornire și auto-testarea	41
5.1.6	Citirea senzorilor și multiplexarea I2C	42
5.1.7	Detecția căderilor în firmware	42
5.1.8	Clasificarea activității pe dispozitiv	42
5.1.9	Sincronizarea configurației WiFi	43
5.1.10	Comunicația cu serverul și coada offline	43
5.1.11	Interfața locală	43
5.1.12	Configurarea rețelei	43
5.2	Backend-ul și API-ul REST	44
5.2.1	Injectia de dependențe și accesul la date	44
5.2.2	Autentificare și securitate	44
5.2.3	Serviciile de fundal	44
5.2.4	Contractul cu clientul și optimizări	45
5.2.5	Validarea cererilor	45
5.3	Microserviciul de inteligență artificială	45
5.3.1	Antrenarea modelului	45
5.3.2	Endpoint-urile serviciului	45
5.3.3	Logica hibridă de decizie	45
5.4	Frontend-ul web	46
5.4.1	Comunicarea cu backend-ul	47
5.4.2	Autorizarea pe roluri și accesibilitatea	47
5.4.3	Organizarea interfeței și gestionarea stării	48
5.4.4	Vizualizarea datelor și funcționalitatea PWA	50
5.4.5	Tabloul de bord și paginile aparținătorului	51
5.4.6	Paginile informative și suportul	53
5.4.7	Interfața de administrare	54
5.4.8	Vizualizarea medicului	56
5.5	Sistemul de notificări	57
5.5.1	Temporizarea alertelor	57
5.5.2	Notificările push	57
5.5.3	SMS și email	58
5.5.4	Raportul zilnic	59
5.5.5	Feedback pentru alarme false	59
6	Testare și rezultate	61
6.1	Metodologia de testare	61
6.2	Acuratețea datelor și performanța modelului AI	61
6.3	Latența sistemului	62
6.4	Fiabilitatea alertelor	63
7	Concluzii și perspective	64
7.1	Concluzii	64
7.2	Limitări identificate	64
7.3	Direcții de dezvoltare ulterioară	64
	Bibliografie	66

Listă de figuri

1	Cele trei niveluri ale arhitecturii IoT și corespondența lor în sistemul LifeAlertPlus.	14
2	Dispozitivul portabil LifeAlertPlus.	16
3	Mecanismul hibrid de decizie: senzorii și pragurile personalizate formează un vector de intrare comun, evaluat în paralel de modelul Random Forest și de setul de reguli medicale; predicțiile sunt combinate în stratul de fuziune, care produce simultan starea discretă și health score-ul.	18
4	Livrarea alertei prin trei canale complementare asigură redundanța mecanismului de notificare.	20
5	Arhitectura generală a sistemului LifeAlertPlus, cu backend-ul în centru și serviciile externe (Twilio, SMTP) reprezentate prin contururi punctate.	27
6	Diagrama de clase simplificată a aplicației LifeAlertPlus, organizată pe trei straturi: API (controlere), Application (servicii) și Domain (entități). Săgețile reprezintă direcția dependențelor între straturi.	29
7	Diagrama entitate-relație a bazei de date LifeAlertPlus. Relația centrală <i>mulți-la-mulți</i> dintre User și Monitored se realizează prin tabela de legătură UserMonitored. Toate entitățile dependente sunt legate <i>unu-la-mulți</i> de Monitored, iar relația punctată indică o legătură opțională (un destinatar pentru o notificare).	31
8	Predicțiile de tendințe afișate în pagina pacientului: etichetă, valoare curentă, rată de variație și timp estimat până la prag.	34
9	Fluxul de declanșare și escaladare a unei alerte.	35
10	Mașina de stări a detectorului de căderi rulat pe dispozitiv.	36
11	Fluxul de sincronizare a configurației WiFi între aplicația web și dispozitivul fizic.	37
12	Schema de conexiuni a dispozitivului portabil.	40
13	Pagina de autentificare a aplicației LifeAlert+: câmpuri pentru email și parolă, plus butonul de autentificare prin Google.	46
14	Pagina de înregistrare: utilizatorul completează datele personale, email-ul, numărul de telefon și parola.	47
15	Asistentul conversațional integrat răspunde la întrebări despre utilizarea aplicației și despre praguri / afecțiuni medicale.	48
16	Pagina cu persoanele monitorizate: cardul rezumă vitalele curente (puls, SpO ₂ , temperatură), starea (Stabil / Alertă / Verificare necesară), poziția GPS și momentul ultimei măsurători.	48
17	Modalul de editare a profilului unei persoane monitorizate, cu pragurile personalizate pentru ritm cardiac, temperatură și SpO ₂ , frecvența de actualizare și perioada de retenție.	49
18	Modalul de configurare a rețelelor WiFi pentru dispozitivul portabil al pacientului.	49
19	Graficele zilnice de puls și temperatură, cu tooltip-ul activ la hover (afișează valoarea, de exemplu 82 bpm la 19:16, și ora) și scrollbar orizontal sincronizat sub fiecare grafic.	50
20	Vizualizarea pe hartă a poziției curente a pacientului în pagina sa de detalii.	51
21	Tabloul de bord al aparținătorului, cu statistici de ansamblu și carduri pentru persoanele recent actualizate.	51
22	Pagina de notificări, cu filtrare după tip și marcarea notificărilor citite.	52
23	Pagina de istoric al măsurătorilor, cu afișarea paginată a valorilor brute.	52
24	Pagina de profil a utilizatorului.	53

25	Pagina de setări a contului, cu preferințele de limbă, notificare și pragurile vitale implicite.	53
26	Pagina „Despre noi”, cu misiunea, problema adresată și caracteristicile proiectului.	54
27	Pagina de suport, cu datele de contact, lista de verificări și întrebările frecvente.	54
28	Tabloul de bord de administrare, cu statisticile agregate la nivelul întregului sistem.	55
29	Jurnalul de activitate (evenimente de cont și de sistem), cu filtrare și căutare.	55
30	Jurnalul de erori al backend-ului, cu filtrare după nivel (eroare, avertisment, informativ).	56
31	Vizualizarea în regim de citire a datelor pacientului, disponibilă medicului invitat.	56
32	Pagina de setări pentru notificări: comutatoarele permit activarea independentă a email, push, SMS și raport zilnic.	57
33	Notificare push afișată în interfața web prin SignalR, imediat ce alerta este declanșată pe server.	58
34	Alertă livrată prin SMS pe telefonul aparținătorului, prin platforma Twilio.	58
35	Email de alertă cu <i>template</i> HTML dedicat, livrat printr-un server SMTP configurat în aplicație.	59
36	Matricea de confuzie a modelului Random Forest.	62
37	Importanța parametrilor de intrare (analiză SHAP).	62

Listă de tabele

1	Comparație între soluțiile existente și limitările identificate.	24
2	Ponderile contribuțiilor în calculul health score-ului.	33
3	Maparea pinilor GPIO ai microcontrollerului ESP32-C3 Mini.	39
4	Rezultatele validării încrucișate stratificate în 5 falduri.	61

Listă de abrevieri

AI	Artificial Intelligence, în traducere inteligență artificială
API	Application Programming Interface, interfață de programare a aplicațiilor
AUC-ROC	Area Under the ROC Curve, aria de sub curba ROC
AV	Alură ventriculară (frecvență cardiacă)
BIDMC	Beth Israel Deaconess Medical Center, instituție sursă a unui set public de date fiziologice
BLE	Bluetooth Low Energy
bpm	Bătăi pe minut (unitate pentru frecvența cardiacă)
CI/CD	Continuous Integration / Continuous Delivery, adică integrare și livrare continuă
CORS	Cross-Origin Resource Sharing
CPaaS	Communication Platform as a Service, platformă de comunicație în cloud
DTO	Data Transfer Object, obiect de transfer de date
ECG	Electrocardiogramă
EDA	Electrodermal Activity, activitate electrodermală
ESP-IDF	Espressif IoT Development Framework, mediul de dezvoltare pentru microcontrollere Espressif
FreeRTOS	Free Real-Time Operating System, sistem de operare în timp real
GPIO	General-Purpose Input/Output, pin de intrare/ieșire de uz general
GPS	Global Positioning System, sistem de poziționare globală
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure, adică HTTP securizat
I2C	Inter-Integrated Circuit, magistrală de comunicație serială
IMU	Inertial Measurement Unit, unitate de măsurare inerțială
IoMT	Internet of Medical Things, internetul dispozitivelor medicale
IoT	Internet of Things, internetul lucrurilor
JSON	JavaScript Object Notation, format de schimb de date
JWT	JSON Web Token
LED	Light-Emitting Diode, diodă electroluminescentă
LiPo	Lithium Polymer, acumulator litiu-polimer
MQTT	Message Queuing Telemetry Transport, protocol de comunicație IoT
NMEA	National Marine Electronics Association, standard de format pentru datele GPS
NVS	Non-Volatile Storage, memorie nevolatilă
OAuth	Open Authorization, protocol de autorizare delegată
OLED	Organic Light-Emitting Diode, ecran cu diode electroluminescente organice
ORM	Object-Relational Mapper, mapare obiect-relațional

PERS	Personal Emergency Response System, sistem de alertă personală medicală
PPG	Photoplethysmography, fotopletismografie
PWA	Progressive Web Application, aplicație web progresivă
REST	Representational State Transfer, stil arhitectural pentru API-uri web
RFID	Radio-Frequency Identification, identificare prin radiofrecvență
RISC-V	Reduced Instruction Set Computer, arhitectură deschisă de procesor
ROC	Receiver Operating Characteristic, curbă de evaluare a clasificatorilor
SHAP	SHapley Additive exPlanations, metodă de explicare a modelelor de inteligență artificială
SMS	Short Message Service, serviciu de mesaje text
SMTP	Simple Mail Transfer Protocol, protocol de trimitere a email-urilor
SPA	Single-Page Application, aplicație web cu pagină unică
SPI	Serial Peripheral Interface, magistrală de comunicație serială
SpO₂	Saturația periferică a oxigenului în sânge
TLS	Transport Layer Security, protocol de securizare a comunicației
TTF	Time To First Fix, timpul până la primul fix GPS
UART	Universal Asynchronous Receiver-Transmitter, interfață de comunicație serială
USB-C	Universal Serial Bus Type-C
WiFi	Tehnologie de rețea locală fără fir (standard IEEE 802.11)

1 Introducere

1.1 Context actual

Transformarea digitală a sistemelor de sănătate reprezintă una dintre tendințele cele mai semnificative ale ultimului deceniu. La nivel global, numărul persoanelor diagnosticate cu afecțiuni cronice precum diabetul, hipertensiunea sau bolile cardiovasculare depășește 1,9 miliarde, potrivit datelor Organizației Mondiale a Sănătății (2023). În România, aproximativ 35% din populație suferă de cel puțin o boală cronică, un procentaj care pune o presiune considerabilă asupra sistemului medical tradițional, bazat în principal pe consultații periodice și internări.

În acest context, tehnologiile Internet of Things (IoT) oferă o alternativă viabilă: monitorizarea continuă a pacienților în afara mediului spitalicesc, prin intermediul senzorilor portabili capabili să măsoare parametri vitali precum ritmul cardiac, saturația oxigenului în sânge (SpO_2) și temperatura corporală, dar și să detecteze, prin senzori de mișcare, evenimente critice precum căderile, care sunt frecvente și potențial grave în cazul persoanelor vârstnice. Datele colectate sunt transmise în timp real către platforme software care le procesează, le stochează și le analizează, permițând intervenția rapidă în situații critice.

Această abordare reduce costurile de spitalizare, crește autonomia pacientului și îmbunătățește calitatea actului medical prin accesul la date obiective și continue.

1.2 Motivația alegerii temei

Alegerea acestei teme a fost motivată de dorința de a dezvolta o soluție tehnologică cu impact direct și măsurabil asupra calității vieții utilizatorilor. Sistemele de monitorizare existente sunt adesea costisitoare, greu de configurat sau limitate la mediul clinic, ceea ce le face inaccesibile pentru utilizatorul obișnuit.

Provocarea de a integra mai multe tehnologii moderne, precum senzori IoT, comunicație wireless, backend scalabil, aplicație web și mecanisme de notificare prin SMS, email și notificări push, a reprezentat un factor motivant suplimentar. Complexitatea tehnică a proiectului oferă oportunitatea aplicării practice a cunoștințelor acumulate pe parcursul studiilor, într-un scenariu cu relevanță reală.

Nu în ultimul rând, posibilitatea de a defini și implementa un mecanism propriu de evaluare a stării de sănătate a utilizatorului, denumit în lucrare *health score*, a constituit o motivație importantă, reprezentând o contribuție originală față de soluțiile existente.

1.3 Scopul lucrării

Scopul lucrării este de a demonstra că un sistem IoT de monitorizare medicală accesibil, fiabil și scalabil poate fi proiectat și implementat cu tehnologii open-source și hardware de uz general, fără a necesita infrastructură specializată de nivel clinic.

Dincolo de aspectul tehnic, lucrarea urmărește să arate că o astfel de soluție poate contribui la prevenirea complicațiilor medicale prin detectarea timpurie a anomaliilor și notificarea automată a persoanelor responsabile, fie ele aparținători, medici sau servicii de urgență. În situațiile critice, intervenția rapidă este sprijinită de localizarea GPS a pacientului și de un buton de panică prin care acesta poate solicita direct ajutor.

1.4 Obiective

Pentru realizarea scopului propus, au fost stabilite următoarele obiective:

O1: proiectarea și implementarea unui subsistem de achiziție a datelor de la senzori biomedicali (ritm cardiac, SpO₂, temperatură corporală) și de la un senzor inerțial pentru detectarea mișcării și a căderilor, precum și a poziției geografice prin GPS, cu transmitere în timp real către un server centralizat.

O2: dezvoltarea unei aplicații web pentru vizualizarea datelor colectate, accesibilă atât pacienților, cât și persoanelor responsabile de monitorizare.

O3: definirea și implementarea unui mecanism inteligent de evaluare a stării de sănătate, care combină un model de învățare automată pentru clasificarea stării pacientului cu un indice sintetic original (health score), bazat pe corelarea mai multor parametri vitali și pe praguri de alertă personalizabile.

O4: integrarea unui sistem de notificări multi-canal (SMS, email, notificări push) care să fie declanșat automat atunci când health score-ul coboară sub un prag critic.

O5: asigurarea funcționării sistemului în timp real, cu latență minimă între achiziția datelor și afișarea/alertarea acestora.

O6: proiectarea unei arhitecturi modulare și scalabile, capabile să susțină un număr crescut de utilizatori și dispozitive fără modificări structurale majore.

1.5 Contribuții personale

Lucrarea de față aduce următoarele contribuții originale:

Arhitectura sistemului: a fost proiectată integral de autor, alegând o structură modulară care separă clar nivelul de achiziție (dispozitiv IoT), nivelul de procesare (backend) și nivelul de prezentare (frontend), facilitând astfel întreținerea și extinderea ulterioară.

Mecanismul health score: reprezintă contribuția cu cel mai înalt grad de originalitate. Spre deosebire de sistemele existente care alertează pe baza depășirii unui singur prag (ex: AV > 100 bpm), health score-ul propus corelează simultan mai mulți parametri vitali, ponderați în funcție de severitate, producând un indice unic al stării de sănătate, mai expresiv și mai puțin predispus la alerte false.

Modelul de inteligență artificială: a fost antrenat și integrat un model de învățare automată (Random Forest) pentru clasificarea automată a stării de sănătate în categoriile normal, alertă și critic. Modelul este combinat cu un set de reguli medicale într-un mecanism hibrid de decizie și este expus printr-un microserviciu dedicat.

Implementarea backend și frontend: întreaga aplicație software a fost dezvoltată de autor, inclusiv API-ul REST pentru comunicarea cu dispozitivele IoT, baza de date pentru stocarea istoricului și interfața web pentru vizualizare.

Integrarea senzorilor: configurarea și calibrarea senzorilor pentru achiziția corectă a datelor biomedicale, precum și implementarea protocolului de comunicație dintre dispozitiv și server.

Stocarea locală și sincronizarea datelor: pentru a asigura continuitatea înregistrării chiar și în absența conexiunii la internet, dispozitivul stochează temporar măsurătorile într-o coadă locală și le sincronizează automat cu serverul la revenirea semnalului.

Sistemul de alertare multi-canal: proiectarea logicii de declanșare a notificărilor și integrarea serviciilor externe pentru livrarea acestora prin SMS, email și notificări push.

1.6 Structura lucrării

Lucrarea este organizată în șapte capitole, fiecare tratând un aspect distinct al dezvoltării sistemului.

Capitolul II prezintă fundamentele teoretice necesare înțelegerii lucrării: concepte IoT, protocoale de comunicație, senzori biomedicali și tehnologii software utilizate.

Capitolul III analizează critic soluțiile existente pe piață și în literatura de specialitate, identificând limitările acestora și justificând abordarea propusă.

Capitolul IV descrie proiectarea sistemului, anume arhitectura generală, fluxul de date, structura bazei de date și logica mecanismului health score.

Capitolul V detaliază implementarea efectivă a fiecărei componente: firmware-ul dispozitivului IoT, backend-ul, frontend-ul și sistemul de notificări.

Capitolul VI prezintă metodologia de testare și rezultatele obținute, cu accent pe acuratețea datelor, latența sistemului și fiabilitatea alertelor.

Capitolul VII formulează concluziile lucrării, limitările identificate și direcțiile de dezvoltare ulterioară.

2 Fundamente teoretice și tehnologii

2.1 Internet of Things

Termenul Internet of Things (IoT) descrie un ecosistem de dispozitive fizice echipate cu senzori, procesoare și module de comunicație, capabile să colecteze și să schimbe date prin intermediul rețelelor de comunicații, fără a necesita intervenție umană directă. Conceptul a fost introdus pentru prima dată de Kevin Ashton în 1999, în contextul identificării prin radiofrecvență (RFID), și s-a extins ulterior pentru a descrie orice obiect conectat la internet capabil să interacționeze cu mediul înconjurător.

Din punct de vedere arhitectural, un sistem IoT este organizat pe trei niveluri principale. Primul nivel, numit nivelul de percepție, cuprinde dispozitivele fizice și senzorii responsabili de achiziția datelor din mediu. Al doilea nivel, nivelul de rețea, asigură transmiterea datelor colectate către infrastructura de procesare, utilizând protocoale precum WiFi, Bluetooth Low Energy (BLE), MQTT sau HTTP. Al treilea nivel, nivelul de aplicație, include platformele software care procesează, stochează și prezintă datele utilizatorilor finali.

În domeniul medical, IoT a generat un subdomeniu distinct, denumit Internet of Medical Things (IoMT), care cuprinde dispozitive precum brățări de monitorizare, pulsioximetre portabile, glucometre conectate și sisteme de monitorizare a semnelor vitale. Potrivit unui raport Fortune Business Insights din 2023, piața globală IoMT a depășit 50 de miliarde de dolari și este estimată să crească la peste 180 de miliarde până în 2030, reflectând cererea crescândă pentru soluții de monitorizare continuă în afara mediului clinic.

Un aspect critic în sistemele IoT medicale este fiabilitatea comunicației. Spre deosebire de aplicațiile industriale sau de automatizare casnică, întreruperea transmisiei datelor într-un sistem medical poate avea consecințe directe asupra siguranței pacientului. Din acest motiv, arhitecturile IoMT includ în mod obișnuit mecanisme de stocare locală temporară a datelor, care asigură continuitatea înregistrării chiar și în absența conectivității la rețea, cu sincronizare automată la revenirea conexiunii.

În cadrul sistemului dezvoltat în această lucrare, arhitectura IoT este implementată pe trei niveluri corespunzătoare: un dispozitiv portabil bazat pe microcontrollerul ESP32-C3 Mini în rolul de nod de percepție, comunicația WiFi și BLE în rolul nivelului de rețea, și o aplicație web deployată pe Azure în rolul nivelului de aplicație. Dat fiind că sistemul este destinat persoanelor vârstnice, s-a acordat o atenție deosebită simplificării interacțiunii cu dispozitivul, configurarea tehnică fiind realizată o singură dată de aparținător, iar funcționarea ulterioară fiind complet transparentă pentru pacient. Corespondența între cele trei niveluri teoretice și componentele concrete ale sistemului este ilustrată în figura 1.

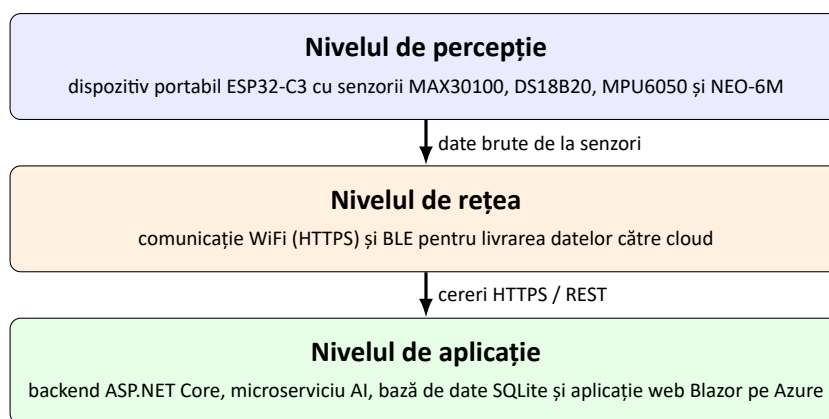


Figura 1: Cele trei niveluri ale arhitecturii IoT și corespondența lor în sistemul LifeAlertPlus.

2.2 Senzori și dispozitive

2.2.1 Microcontrollerul ESP32-C3 Mini

ESP32-C3 Mini este un microcontroller de dimensiuni reduse, produs de Espressif Systems, bazat pe arhitectura RISC-V pe 32 de biți. Dispune de conectivitate WiFi 802.11 b/g/n și Bluetooth 5.0 Low Energy, ceea ce îl face potrivit pentru aplicații IoT portabile care necesită atât comunicație locală, cât și acces la internet. Consumul redus de energie, dimensiunile compacte și suportul pentru protocoale de comunicație standard precum I2C, SPI și UART îl recomandă ca platformă centrală pentru un dispozitiv de tip brățară medicală.

În cadrul sistemului, ESP32-C3 Mini îndeplinește rolul de unitate centrală de procesare a dispozitivului portabil: colectează datele de la toți senzorii, le procesează local, le afișează pe ecranul OLED și le transmite către serverul din cloud prin WiFi sau, în absența acestuia, prin BLE către telefonul aparținătorului.

2.2.2 Senzorul MAX30100 pentru puls și saturație de oxigen

MAX30100 este un senzor optic integrat, produs de Maxim Integrated, destinat măsurării neinvazive a frecvenței cardiace și a saturației oxigenului în sânge (SpO_2). Funcționarea sa se bazează pe principiul **fotoplețismografiei (PPG)**: senzorul emite lumină în spectrul roșu și infraroșu prin țesutul cutanat, iar un fotodetector măsoară variațiile de absorbție cauzate de pulsațiile sângelui. Diferența de absorbție dintre hemoglobina oxigenată și cea dezoxigenată permite calcularea SpO_2 .

Comunicația cu microcontrollerul se realizează prin protocolul **I2C**, la o adresă fixă de 0x57. Tensiunea de alimentare este de 1.8V pentru nucleul digital și 3.3V pentru LED-uri, necesitând atenție la proiectarea schemei electrice. Valorile normale ale parametrilor măsurați sunt între 60 și 100 bătăi pe minut pentru frecvența cardiacă și între 95% și 100% pentru SpO_2 , valorile sub 90% indicând o hipoxemie potențial critică.

2.2.3 Senzorul DS18B20 pentru temperatura corporală

DS18B20 este un senzor digital de temperatură produs de Dallas Semiconductor (Maxim Integrated), cu o rezoluție configurabilă între 9 și 12 biți și o precizie de $\pm 0.5^\circ\text{C}$ în intervalul $0-70^\circ\text{C}$. Comunicația se realizează prin protocolul **1-Wire**, care permite conectarea mai multor senzori pe același fir de date, fiecare identificat printr-un cod unic de 64 de biți. Intervalul de măsurare este de -55°C până la $+125^\circ\text{C}$, acoperind cu generozitate atât intervalul temperaturilor corporale normale (36.1°C până la 37.2°C), cât și pe cel al temperaturilor febrile (peste 38°C).

Un avantaj important pentru aplicația de față este că DS18B20 poate fi utilizat în variantă impermeabilă, potrivită pentru contact cu pielea, și are un consum extrem de redus în modul de așteptare.

2.2.4 Senzorul MPU6050 pentru mișcare și accelerație

MPU6050, produs de InvenSense, este o unitate de măsurare inerțială (IMU) care integrează un accelerometru triaxial și un giroscop triaxial într-un singur circuit, comunicând prin **I2C** la adresele 0x68 sau 0x69. Accelerometrul măsoară accelerația pe axe X, Y și Z, iar giroscopul măsoară viteza unghiulară, permițând împreună determinarea orientării și a tipului de mișcare al purtătorului.

În contextul acestei lucrări, datele furnizate de MPU6050 sunt utilizate în două scopuri distincte. Primul scop este **detectarea activității fizice** (mers, repaus, somn), pentru construirea profilului de activitate zilnică al pacientului. Al doilea scop, cu implicații directe asupra siguranței, este **detectarea căderilor**, un eveniment frecvent și potențial grav în cazul persoanelor vârstnice. Algoritmul de detecție folosește un detector clasic bazat pe praguri aplicate magnitudinii accelerației: o cădere reală combină o scurtă perioadă de *free-fall* (magnitudine sub aproximativ 0,5 g), un *impact* brusc (magnitudine peste 2,5 g) și o perioadă ulterioară de *imobilitate* a pacientului (variații mici în jurul valorii de 1 g). Implementarea concretă a acestui algoritm, ca task FreeRTOS dedicat pe microcontroller, este detaliată în subcapitolul 5.2.

Interpretarea datelor brute ale senzorului pentru clasificarea activităților este realizată de un modul AI descris în subcapitolul 2.3.

2.2.5 Modulul GPS NEO-6M

NEO-6M este un modul GPS produs de u-blox, capabil să determine poziția geografică cu o precizie de aproximativ 2.5 metri în condiții favorabile. Comunicația cu microcontrollerul se realizează prin **UART**, la o rată de 9600 baud implicit, transmițând date în format NMEA 0183. Timpul de achiziție a primului fix GPS (TTFF) este de aproximativ 27 de secunde la pornire rece și sub 1 secundă la pornire caldă.

În cadrul sistemului, modulul GPS servește la localizarea pacientului în scenarii de alertă, permițând aparținătorului să vizualizeze ultima poziție cunoscută în aplicație. Deoarece GPS-ul are un consum relativ ridicat față de ceilalți senzori, frecvența de interogare este configurabilă, reducând impactul asupra autonomiei bateriei.

2.2.6 Multiplexorul I2C TCA9548A

TCA9548A este un multiplexor I2C pe 8 canale, produs de Texas Instruments, care permite conectarea simultană a mai multor dispozitive I2C cu aceeași adresă la același microcontroller. Această componentă rezolvă conflictele de adresă care apar inevitabil atunci când se conectează mai mulți senzori pe aceeași magistrală I2C; în cazul de față, MAX30100 și MPU6050 utilizează ambele magistrala I2C. Comunicația cu ESP32-C3 se realizează tot prin I2C, la adresa 0x70, iar selectarea canalului activ se face prin scriere software.

2.2.7 Sistemul de alimentare

Alimentarea dispozitivului portabil este asigurată de un acumulator **LiPo 3.7V 1500mAh** (103048 AM Electronics), cu dimensiuni compacte de 10×30×48mm și protecție integrată împotriva supraîncărcării, supradescărcării și scurtcircuitului. Gestionarea încărcării este realizată de modulul **TP4056**, un circuit integrat dedicat încărcării bateriilor Li-Ion/Li-Po prin USB-C, cu curent

de încărcare de 1A și protecție completă a celulei. Conversia tensiunii de la 3.7V la nivelul necesar componentelor este asigurată de convertorul boost **MT3608**, capabil să furnizeze până la 28V la 2A, configurat în acest caz pentru tensiunea de operare a sistemului.

2.2.8 Interfața utilizator pe dispozitiv

Dispozitivul include două elemente de interfață locală. Un **ecran OLED** afișează în timp real informații esențiale precum frecvența cardiacă, SpO_2 , temperatura și starea conexiunii, oferind pacientului sau aparținătorului o confirmare vizuală a funcționării. Două **LED-uri** indică starea sistemului: verde pentru funcționare normală și roșu pentru eroare sau alertă activă.

Interacțiunea fizică cu dispozitivul este limitată la două butoane, în concordanță cu principiul simplității pentru utilizatorul vârstnic: un buton de pornire/oprire și un **buton de panică**, a cărui apăsare declanșează imediat o alertă către aparținător, indiferent de valorile senzorilor.

[Spațiu pentru figură: fotografia dispozitivului portabil asamblat, cu senzorii, ecranul OLED și butoanele vizibile.]

Figura 2: Dispozitivul portabil LifeAlertPlus.

2.3 Prelucrarea datelor și inteligență artificială

Datele brute furnizate de senzorii biomedicali nu pot fi utilizate direct pentru a decide dacă starea pacientului este una normală sau una care necesită intervenție. Valorile individuale, precum o frecvență cardiacă de 105 bpm sau o temperatură de 37,8°C, sunt ambigue atunci când sunt interpretate izolat: ele pot indica fie o stare patologică, fie o reacție fiziologică firească la efort, stres sau temperatura ambientală. Din acest motiv, transformarea datelor în informație utilă din punct de vedere clinic presupune o etapă de prelucrare și o etapă de interpretare, în care intervine inteligența artificială.

În sistemul dezvoltat, prelucrarea datelor este organizată pe două niveluri. Primul nivel, realizat local pe microcontroller, cuprinde filtrarea valorilor evident eronate (citiri în afara intervalelor fizic posibile, generate de contactul imperfect al senzorului cu pielea) și calculul unor mărimi derivate, precum magnitudinea vectorului de accelerație, obținută din componentele pe cele trei axe. Al doilea nivel, realizat pe server, cuprinde analiza propriu-zisă a stării de sănătate, prin combinarea unui model de învățare automată cu un set de reguli medicale deterministe.

2.3.1 Învățarea automată și problema de clasificare

Învățarea automată (machine learning) este subdomeniul inteligenței artificiale care studiază algoritmi capabili să își îmbunătățească performanța pe baza datelor, fără a fi programați explicit pentru fiecare situație posibilă. În cazul de față, evaluarea stării pacientului a fost formulată ca o problemă de **clasificare supervizată**: pornind de la un set de parametri vitali, modelul

trebuie să atribuie pacientului una dintre trei clase, anume NORMAL, ALERT sau CRITICAL. Clasificarea supervizată presupune existența unui set de date de antrenare în care fiecare exemplu este însoțit de eticheta corectă, modelul deducând din aceste exemple o funcție de decizie care poate fi aplicată ulterior unor date noi.

Algoritmul utilizat este **Random Forest**, un model de tip ansamblu (ensemble) format dintr-o mulțime de arbori de decizie antrenați independent pe subseturi aleatorii ale datelor. Predicția finală rezultă din agregarea voturilor tuturor arborilor, ceea ce reduce semnificativ riscul de supraînvățare (overfitting) caracteristic unui arbore de decizie individual. Random Forest a fost preferat datorită robusteții în raport cu zgomotul din date, a capacității de a estima importanța fiecărui parametru de intrare și a faptului că nu necesită un volum de date la fel de mare ca rețelele neuronale, un avantaj practic important într-un domeniu în care datele medicale reale etichetate sunt greu de obținut. Modelul a fost configurat cu 150 de arbori, o adâncime maximă de 10 niveluri și ponderarea claselor, pentru a compensa distribuția lor inegală.

2.3.2 Datele de antrenare și pregătirea lor

Întrucât colectarea unui volum suficient de date medicale reale, etichetate, ridică probleme practice și de confidențialitate, modelul a fost antrenat pe un set de date sintetice, generat pornind de la trei surse publice recunoscute: setul de date BIDMC pentru frecvența cardiacă și saturația oxigenului, un set de date de termometrie pentru temperatura corporală și un set de date provenit de la senzori MPU6050 pentru parametrii de mișcare. Peste valorile extrase din aceste surse a fost adăugat un zgomot gaussian controlat, pentru a obține o variabilitate realistă, și au fost generate suplimentar exemple corespunzătoare stărilor ALERT și CRITICAL, mai rare în datele reale, astfel încât modelul să poată învăța corect și situațiile critice.

Înainte de antrenare, datele au fost supuse unor operații standard de preprocesare. Parametrii proveniți de la accelerometru și giroscop au fost normalizați prin metoda Min-Max, care aduce toate valorile în intervalul $[0, 1]$, împiedicând ca mărimile cu domeniu numeric larg să domine procesul de învățare. Etichetele textuale ale claselor au fost convertite în valori numerice prin codificare (label encoding), iar saturației oxigenului i s-a atribuit o pondere suplimentară, fiind cel mai relevant indicator clinic dintre parametrii monitorizați.

2.3.3 Sistemul hibrid de decizie

O caracteristică importantă a soluției este faptul că decizia finală nu este lăsată exclusiv în seama modelului de învățare automată. Predicția acestuia este combinată cu un set de **reguli medicale deterministe**, derivate din praguri clinice consacrate. Motivul acestei abordări hibride este unul de siguranță: un model de învățare automată, oricât de bine antrenat, poate produce predicții eronate în cazuri rare sau atipice, în timp ce o regulă medicală absolută (de exemplu, o saturație a oxigenului sub un prag critic clasificată automat ca stare CRITICAL) garantează că situațiile fără echivoc sunt tratate corect indiferent de comportamentul modelului. Astfel, modelul AI aduce capacitatea de a recunoaște tipare complexe și corelații subtile între parametri, iar regulile medicale aduc garanții de corectitudine și explicabilitate.

În plus față de starea discretă (NORMAL/ALERT/CRITICAL), sistemul calculează un **health score**, un indice sintetic cu valori între 0 și 100 care exprimă numeric gradul de risc al pacientului la un moment dat. Spre deosebire de starea discretă, health score-ul oferă o măsură continuă, care permite urmărirea tendințelor (o stare care se degradează lent, dar constant) și declanșarea graduală a alertelor. Logica de calcul a acestui indice, precum și modul în care ține cont de pragurile personalizate ale fiecărui pacient și de afecțiunile preexistente ale acestuia, sunt detaliate în capitolul de proiectare a sistemului. Schema mecanismului hibrid de decizie este

prezentată în figura 3.

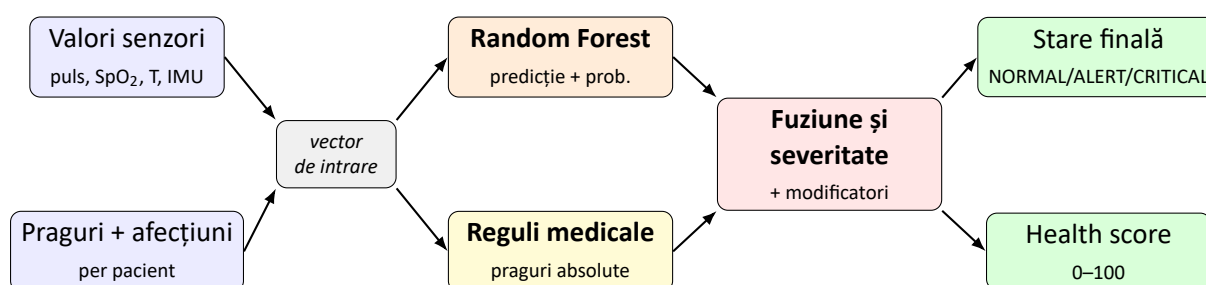


Figura 3: Mecanismul hibrid de decizie: senzorii și pragurile personalizate formează un vector de intrare comun, evaluat în paralel de modelul Random Forest și de setul de reguli medicale; predicțiile sunt combinate în stratul de fuziune, care produce simultan starea discretă și health score-ul.

2.3.4 Explicabilitatea și evaluarea modelului

Într-un domeniu cu implicații medicale, un model care produce predicții corecte, dar de neînțeles, este insuficient: este necesar ca decizia să poată fi explicată. Din acest motiv, modelul a fost analizat folosind metoda **SHAP (SHapley Additive exPlanations)**, o tehnică ce cuantifică contribuția fiecărui parametru de intrare la o predicție individuală. Analiza SHAP a confirmat că saturația oxigenului, frecvența cardiacă și temperatura corporală sunt factorii dominanți în decizia modelului, în concordanță cu criteriile clinice de triaj.

Performanța modelului a fost estimată prin **validare încrucișată stratificată în 5 falduri** (stratified 5-fold cross-validation), o metodă prin care setul de date este împărțit repetat în sub-seturi de antrenare și testare, păstrând în fiecare fald aceeași distribuție a claselor. Această abordare oferă o estimare mai realistă a capacității de generalizare decât o singură împărțire antrenare/testare și permite, totodată, verificarea absenței supraînvățării prin compararea performanței pe datele de antrenare cu cea pe datele de testare. Au fost urmărite metrice precum acuratețea, scorul F1 și aria de sub curba ROC.

2.3.5 Clasificarea activității și arhitectura de tip microserviciu

Pe lângă evaluarea stării de sănătate, datele de mișcare furnizate de senzorul MPU6050 sunt prelucrate **pe dispozitiv** pentru a clasifica activitatea purtătorului ca *moving* sau *stationary*, și pentru a contribui la detectarea căderilor. Eticheta de activitate este derivată din acumularea abaterii magnitudinii accelerației față de gravitație ($|\vec{a}| - 1\text{ g}$) peste fereastra de 5 s între două POST-uri: media și maximul abaterii peste această fereastră sunt comparate cu praguri empirice (0,05 g pentru media de mișcare susținută, 0,20 g pentru un vârf izolat), iar eticheta rezultată este inclusă direct în payload-ul JSON trimis backend-ului. Această clasificare reutilizează exact aceleași sample-uri pe care fall-detector-ul le citește la 50 Hz, fără overhead suplimentar. Pe baza acestei etichete, persistată pentru fiecare măsurătoare, backend-ul agregă **profilul de activitate zilnică** (rata de mișcare și probabilitatea de somn per oră, pe o fereastră de 14 zile) și detectează două tipuri de anomalii comportamentale: *inactivitate prelungită* (peste 15 minute fără mișcare într-o oră în care pacientul e de obicei activ) și *activitate nocturnă* (mișcare detectată într-o oră în care pacientul doarme de obicei).

Din punct de vedere arhitectural, componenta de inteligență artificială propriu-zisă (clasificarea stării vitale în NORMAL/ALERT/CRITICAL și calculul health score-ului) este implementată ca un

microserviciu independent, scris în limbajul Python și expus printr-o interfață REST. Această separare a fost aleasă deliberat: ecosistemul Python oferă cele mai mature biblioteci de învățare automată, iar izolarea modulului AI într-un serviciu propriu permite actualizarea, înlocuirea sau scalarea modelului fără a afecta restul aplicației. Backend-ul aplicației comunică cu microserviciul AI prin cereri HTTP, transmițând valorile senzorilor și primind în răspuns starea prezisă, health score-ul și o explicație textuală a deciziei.

2.4 Sisteme de alertare

Un sistem de alertare reprezintă ansamblul de mecanisme prin care un sistem informatic semnalează automat apariția unei situații care necesită atenție. În contextul monitorizării medicale, sistemul de alertare este componenta care transformă o anomalie detectată într-o acțiune concretă: informarea unei persoane capabile să intervină. Funcționarea sa presupune trei etape succesive, anume detectarea evenimentului, decizia de a alerta și livrarea efectivă a notificării.

2.4.1 Fiabilitate și problema alertelor false

Cerința fundamentală a unui sistem de alertare medicală este fiabilitatea, înțeleasă în dublu sens: sistemul trebuie să semnaleze toate situațiile reale de risc (sensibilitate ridicată), dar trebuie, în același timp, să evite alarmele nejustificate (specificitate ridicată). Un sistem care generează frecvent alerte false produce fenomenul cunoscut sub numele de *alert fatigue*, adică obișnuirea destinatarului cu notificările, urmată de ignorarea acestora, inclusiv a celor reale. Pentru o aplicație destinată siguranței persoanelor vârstnice, acest efect ar anula însuși scopul sistemului.

Din acest motiv, sistemele de alertare moderne includ mai multe mecanisme de reducere a alertelor false. **Pragul de persistență** impune ca o situație anormală să se mențină o durată minimă de timp înainte de a declanșa o notificare, eliminând astfel valorile tranzitorii cauzate de un contact imperfect al senzorului. Mecanismul de **cooldown** (perioadă de inhibare) împiedică transmiterea repetată a aceleiași alerte într-un interval scurt, evitând inundarea destinatarului cu mesaje. **Escaladarea** presupune trecerea succesivă la canale de comunicație mai insistente atunci când o alertă nu este confirmată; de exemplu, o notificare push poate fi urmată de un SMS dacă situația persistă.

2.4.2 Alertarea multi-canal

Niciun canal de comunicație nu este, singur, suficient de fiabil pentru un scenariu medical. Din acest motiv, soluția propusă utilizează trei canale complementare, fiecare cu caracteristici proprii.

SMS: mesajele text reprezintă canalul cu cea mai mare acoperire, întrucât nu necesită ca destinatarul să dețină un smartphone, o aplicație instalată sau o conexiune la internet. SMS-ul este deosebit de potrivit pentru notificarea aparținătorilor în orice condiții. Livrarea SMS-urilor se realizează prin intermediul unei platforme de comunicație în cloud (CPaaS, adică Communication Platform as a Service), care expune o interfață de programare prin care aplicația poate trimite mesaje fără a gestiona direct infrastructura de telefonie.

Email: poșta electronică este potrivită pentru notificări mai puțin urgente și pentru transmiterea de conținut bogat: rapoarte, grafice, atașamente. Spre deosebire de SMS, livrarea email-ului nu este garantată în timp real și poate fi întârziată sau filtrată ca spam, motiv pentru care este utilizată ca un canal complementar, nu ca singura cale de alertare.

Notificările push: acestea sunt mesaje livrate instantaneu în interfața aplicației web, atâta

vreme cât aceasta este deschisă. Notificările push oferă cea mai mică latență dintre toate canalele și sunt ideale pentru actualizarea în timp real a tabloului de bord al aparținătorului. Ele depind însă de existența unei conexiuni active, motiv pentru care sunt dublate de SMS în situațiile critice.

Combinarea celor trei canale asigură redundanță: dacă un canal eșuează sau nu este observat, un altul preia rolul de informare, crescând probabilitatea ca aparținătorul să fie alertat la timp. Fluxul comparativ al celor trei canale este sintetizat în figura 4.

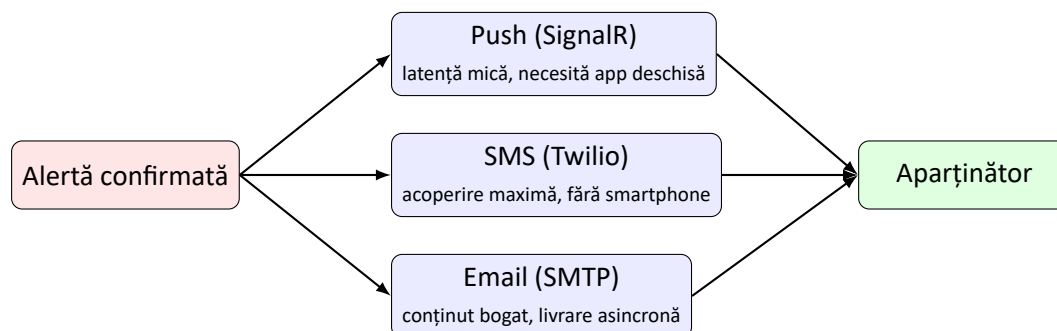


Figura 4: Livrarea alertei prin trei canale complementare asigură redundanța mecanismului de notificare.

2.5 Tehnologii utilizate (backend, frontend, Docker etc.)

Realizarea unui sistem IoT funcțional, dincolo de componenta hardware, presupune integrarea unui set coerent de tehnologii software. Această secțiune prezintă pe scurt tehnologiile pe care se sprijină aplicația, grupate pe niveluri arhitecturale.

2.5.1 Backend-ul aplicației

Backend-ul reprezintă nivelul de procesare al sistemului și a fost dezvoltat folosind **ASP.NET Core**, un framework open-source și cross-platform dezvoltat de Microsoft pentru construirea de aplicații web și interfețe de programare, utilizând limbajul C#. Backend-ul expune un **API REST**, un stil arhitectural bazat pe protocolul HTTP, în care resursele sunt accesate prin verbe standard (GET, POST, PUT, DELETE), iar datele sunt schimbate în format JSON. Prin acest API comunică atât dispozitivele IoT, care transmit măsurătorile, cât și aplicația web, care le consultă.

Codul backend-ului este organizat conform principiilor unei **arhitecturi stratificate (clean architecture)**, separat în mai multe straturi cu responsabilități distincte: stratul de domeniu, care conține entitățile fundamentale; stratul de aplicație, care conține logica de business; stratul de infrastructură, responsabil cu accesul la date și cu serviciile externe; și stratul de prezentare a API-ului, care conține controlerele. Această separare facilitează testarea, întreținerea și extinderea ulterioară a sistemului.

Persistența datelor este asigurată de o bază de date relațională **SQLite**, un sistem de baze de date încorporat, care stochează întreaga bază într-un singur fișier și nu necesită un server dedicat. Accesul la baza de date se realizează prin intermediul unui ORM (Object-Relational Mapper), anume **Entity Framework Core**, care mapează automat clasele din cod pe tabelele bazei de date și permite formularea interogărilor direct în limbajul C#.

2.5.2 Securitatea

Întrucât sistemul gestionează date medicale, securitatea reprezintă o cerință esențială. Autentificarea utilizatorilor se bazează pe **JWT (JSON Web Token)**, un mecanism de autentificare fără stare în care, după autentificare, serverul emite un token semnat criptografic ce conține informații despre utilizator și care este prezentat la fiecare cerere ulterioară. Aplicația permite, de asemenea, autentificarea prin **OAuth 2.0** folosind un cont Google, un protocol standard de autorizare delegată care elimină necesitatea gestionării unei parole suplimentare. Parolele utilizatorilor care aleg înregistrarea clasică sunt stocate sub formă de hash generat cu algoritmul **BCrypt**, conceput special pentru a fi lent și rezistent la atacuri de tip forță brută.

2.5.3 Comunicarea în timp real

Pentru actualizarea instantanee a interfeței și pentru livrarea notificărilor push, sistemul folosește **SignalR**, o bibliotecă care permite comunicarea bidirecțională în timp real între server și client. SignalR abstractizează protocolul WebSocket, menținând o conexiune permanentă prin care serverul poate transmite mesaje către client fără ca acesta să le ceară explicit.

2.5.4 Frontend-ul aplicației

Interfața web a fost dezvoltată folosind **Blazor WebAssembly**, un framework care permite scrierea aplicațiilor web interactive în limbajul C#, fără a fi necesar JavaScript pentru logica aplicației. Codul C# este compilat în **WebAssembly**, un format binar de instrucțiuni care rulează direct în browser, la o viteză apropiată de cea nativă. Aplicația este o **SPA (Single-Page Application)**, adică o aplicație web care se încarcă o singură dată și care actualizează dinamic conținutul, fără reîncărcarea completă a paginii.

Aplicația este, în plus, o **PWA (Progressive Web Application)**: include un service worker care permite funcționarea parțială în lipsa conexiunii la internet, un fișier manifest care permite instalarea aplicației pe dispozitiv ca o aplicație nativă și o pagină dedicată pentru modul offline.

2.5.5 Containerizarea și implementarea

Pentru asigurarea consistenței între mediul de dezvoltare și cel de producție, componentele aplicației sunt împachetate folosind **Docker**, o tehnologie de containerizare care încapsulează o aplicație împreună cu toate dependențele sale într-o imagine portabilă. Cele trei componente principale ale sistemului, anume microserviciul AI, backend-ul și frontend-ul, sunt orchestrate prin **Docker Compose**, un instrument care permite definirea și pornirea coordonată a mai multor containere. Servirea fișierelor statice ale interfeței web este realizată printr-un server **nginx**.

Procesul de construire, testare și publicare a aplicației este automatizat printr-un flux de **integritate și livrare continuă (CI/CD)** configurat cu GitHub Actions, iar sistemul este deployat pe infrastructura cloud **Microsoft Azure**, asigurând disponibilitatea și accesul de la distanță.

3 Analiza soluțiilor existente

3.1 Prezentarea soluțiilor actuale

Piața dispozitivelor de monitorizare medicală portabilă a cunoscut o expansiune semnificativă în ultimul deceniu, atât din perspectiva soluțiilor comerciale de masă, cât și a sistemelor dedicate mediului clinic sau îngrijirii persoanelor vârstnice. În cele ce urmează sunt prezentate cele mai relevante categorii de soluții existente, analizate din perspectiva funcționalității, a publicului țintă și a tehnologiilor utilizate.

3.1.1 Fitbit Sense 2 / Google Pixel Watch

Fitbit, achiziționat de Google în 2021, reprezintă unul dintre cele mai cunoscute ecosisteme de monitorizare a sănătății destinate publicului larg. Dispozitivele din gama Sense monitorizează frecvența cardiacă, SpO₂, temperatura pielii, nivelul de stres prin electrodermal activity (EDA) și calitatea somnului. Datele sunt sincronizate prin BLE cu aplicația mobilă Fitbit, disponibilă pe Android și iOS, și stocate în cloud-ul Google.

Deși ecosistemul Fitbit este matur și bine documentat, el este proiectat pentru utilizatori activi, orientați spre fitness și auto-monitorizare. Interfața aplicației presupune un nivel de familiarizare digitală incompatibil cu profilul persoanelor vârstnice, iar sistemul de alertare este limitat la notificări generale, fără posibilitatea configurării unor praguri personalizate sau a notificării unui aparținător desemnat.

3.1.2 Apple Watch (Series 9 / Ultra 2)

Apple Watch reprezintă cel mai avansat dispozitiv portabil de monitorizare a sănătății disponibil comercial. Seria 9 include senzori pentru ECG, frecvență cardiacă, SpO₂, temperatură și detecție a căderilor, aceasta din urmă fiind direct relevantă pentru scenariul persoanelor vârstnice. Funcția Emergency SOS permite contactarea serviciilor de urgență automat după detectarea unei căderi, dacă utilizatorul nu răspunde în intervalul de timp configurat.

Cu toate acestea, Apple Watch necesită un iPhone asociat pentru configurare și funcționare completă, ceea ce presupune că pacientul trebuie să dețină și să opereze un smartphone Apple, o cerință nerealistă pentru majoritatea persoanelor vârstnice din România. Costul ridicat al ecosistemului (dispozitiv + abonament iCloud) reprezintă o barieră suplimentară. Mai mult, platforma este închisă, nepermițând integrarea cu sisteme externe de monitorizare sau exportul structurat al datelor către un medic sau aparținător prin canale personalizate.

3.1.3 Philips Lifeline / Medical Guardian

Philips Lifeline și Medical Guardian reprezintă o categorie distinctă de dispozitive, anume sistemele de alertă personală medicală (PERS, adică Personal Emergency Response Systems), dedicate exclusiv persoanelor vârstnice sau cu mobilitate redusă. Acestea iau forma unor butoane de panică purtate ca pandantiv sau brățară, conectate la o centrală care permite contactarea unui operator uman sau a serviciilor de urgență.

Avantajul principal al acestor sisteme este simplitatea extremă din perspectiva utilizatorului, anume un singur buton fizic. Dezavantajul major este că nu realizează monitorizare continuă a parametrilor vitali. Sistemul reacționează exclusiv la inițiativa pacientului, ceea ce îl face ineficient în scenarii precum pierderea conștienței, căderea cu imobilizare sau degradarea lentă a stării de sănătate, când pacientul este incapabil să apese butonul.

3.1.4 Withings ScanWatch

Withings ScanWatch este un ceas inteligent cu design clasic, care monitorizează frecvența cardiacă, SpO₂, respirația în somn și activitatea fizică. Spre deosebire de Fitbit sau Apple Watch, ScanWatch are un design discret, similar unui ceas tradițional, ceea ce îl face mai acceptabil estetic pentru utilizatorii vârstnici. Autonomia bateriei de până la 30 de zile reprezintă un avantaj semnificativ față de concurență.

Totuși, și Withings ScanWatch este orientat spre auto-monitorizare, fără un sistem dedicat de alertare a aparținătorilor și fără posibilitatea configurării unor praguri personalizate de alertă. Exportul datelor se face prin aplicația Health Mate, care nu oferă funcționalitate de tip dashboard pentru o persoană terță (aparținător).

3.1.5 Soluții academice și de cercetare

Literatura de specialitate propune numeroase sisteme IoT de monitorizare medicală dezvoltate în context academic, bazate frecvent pe platforme Arduino sau ESP32, senzori MAX30100/MAX30102 și protocoale MQTT pentru transmiterea datelor. Aceste sisteme demonstrează viabilitatea tehnică a abordării, însă rămân în stadiul de prototip de laborator, fără interfețe utilizator complete, fără mecanisme de alertare multi-canal și fără considerarea aspectelor practice legate de autonomia bateriei, rezistența mecanică sau ușurința de utilizare pentru persoane fără competențe tehnice.

3.2 Limitări identificate

Analiza soluțiilor prezentate în secțiunea anterioară relevă o serie de limitări comune, grupate în câteva categorii principale.

3.2.1 Inadecvarea pentru utilizatorii vârstnici

Majoritatea soluțiilor comerciale sunt proiectate pentru utilizatori tineri sau de vârstă medie, cu competențe digitale solide. Interfețele complexe, necesitatea unui smartphone asociat și procesele de configurare multi-pas reprezintă bariere reale pentru persoanele vârstnice. Sistemele PERS (Philips Lifeline, Medical Guardian) rezolvă parțial această problemă prin simplitate, dar sacrifică complet monitorizarea continuă.

3.2.2 Lipsa monitorizării continue și a alertării inteligente

Dispozitivele comerciale de tip smartwatch alertează utilizatorul însuși, nu un aparținător desemnat. Scenariul critic, în care pacientul este incapabil să reacționeze, iar aparținătorul rămâne neinformați, nu este acoperit adecvat de niciuna dintre soluțiile analizate. Sistemele PERS depind exclusiv de inițiativa pacientului, iar smartwatch-urile nu oferă un mecanism fiabil de escaladare a alertelor către persoane terțe prin canale multiple (SMS, email, push).

3.2.3 Platforme închise și lipsa personalizării

Soluțiile comerciale funcționează în ecosisteme închise care nu permit integrarea cu sisteme externe, configurarea pragurilor de alertă în funcție de profilul medical individual sau exportul structurat al datelor. Un medic nu poate accesa direct datele unui pacient care folosește Fitbit sau Apple Watch fără ca pacientul să realizeze manual operațiuni de export.

3.2.4 Costul și dependența de abonamente

Atât Apple Watch, cât și sistemele PERS implică costuri recurente, precum abonamente cloud sau taxe lunare pentru servicii de monitorizare umană, care pot fi prohibitive pe termen lung,

în special pentru persoane cu venituri fixe reduse, categorie frecvent suprapusă cu cea a vârstnicilor.

3.2.5 Limitări ale soluțiilor academice

Prototipurile academice demonstrează conceptul tehnic, dar nu adresează aspecte esențiale pentru un sistem funcțional real: interfață utilizator completă, securitatea datelor medicale, mecanisme de recuperare la pierderea conexiunii, autonomia bateriei în condiții reale de utilizare sau scalabilitatea la număr mare de utilizatori.

Tabelul 1 sintetizează comparativ limitările identificate.

Criteriu	FitBit/ Google	Apple Watch	Philips Lifeline	Withings	Soluții academice
Potrivit pentru vârstnici	Parțial	Nu	Da	Parțial	Nu
Monitorizare continuă vitale	Da	Da	Nu	Da	Da
Alertare aparținător multi-canal	Nu	Nu	Parțial	Nu	Nu
Praguri personalizabile	Nu	Nu	Nu	Nu	Parțial
Export date structurat	Limitat	Limitat	Nu	Limitat	Parțial
Fără abonament obligatoriu	Nu	Nu	Nu	Nu	Da
Stocare locală la pierdere semnal	Nu	Nu	Nu	Nu	Nu
Detecție căderi	Nu	Da	Nu	Nu	Nu
Healthscore integrat	Nu	Nu	Nu	Nu	Nu

Tabela 1: Comparatie între soluțiile existente și limitările identificate.

3.3 Poziționarea soluției propuse

Soluția dezvoltată în cadrul acestei lucrări se poziționează într-un spațiu neacoperit adecvat de produsele existente: un sistem de monitorizare continuă a parametrilor vitali, dedicat persoanelor vârstnice, cu alertare automată multi-canal a aparținătorilor și configurare tehnică minimă din partea pacientului.

Față de soluțiile comerciale de tip smartwatch, sistemul propus oferă:

- **Alertare activă a aparținătorului** prin SMS, email și notificări push, fără a depinde de inițiativa pacientului;
- **Praguri de alertă personalizabile** per pacient, configurabile de aparținător din aplicația web;

- **Health score**, un indice sintetic al stării de sănătate bazat pe corelarea mai multor parametri vitali simultan, absent din oricare dintre soluțiile analizate;
- **Stocare locală a datelor** la pierderea conectivității, cu sincronizare automată la revenirea semnalului;
- **Export structurat** al datelor direct către medicul curant, prin email, dintr-un interval selectabil.

Față de sistemele PERS (butoane de panică), soluția propusă adaugă monitorizarea continuă și detecția automată a situațiilor critice, eliminând dependența de capacitatea pacientului de a apăsa un buton în momentul critic.

Față de prototipurile academice, sistemul propus depășește stadiul de demonstrație tehnică prin includerea unei interfețe web complete, a unui sistem de autentificare securizat, a unui mecanism de notificări funcțional și a unei arhitecturi deployate pe infrastructură cloud reală.

Această poziționare confirmă relevanța și originalitatea soluției propuse, care nu încearcă să concureze frontal cu ecosistemele comerciale mature, ci adresează un segment specific și insuficient acoperit: monitorizarea continuă, automată și inteligentă a persoanelor vârstnice, cu implicarea activă a aparținătorului prin tehnologii accesibile și fără costuri recurente semnificative.

4 Proiectarea sistemului

Acest capitol descrie deciziile de proiectare luate înaintea implementării propriu-zise. Sunt prezentate, pe rând, arhitectura de ansamblu a sistemului, componentele care îl alcătuiesc și rolul fiecăreia, structura bazei de date și, în final, fluxurile prin care datele circulă prin sistem, de la achiziția lor pe dispozitiv până la declanșarea unei alerte. Tot aici este detaliată logica mecanismului health score, anunțată în subcapitolul 2.3.

4.1 Arhitectura generală

Sistemul LifeAlertPlus respectă modelul arhitectural pe trei niveluri specific aplicațiilor IoT, prezentat în subcapitolul 2.1. Fiecărui nivel teoretic îi corespunde o componentă concretă: nivelului de percepție îi corespunde dispozitivul portabil bazat pe microcontrollerul ESP32-C3 Mini; nivelului de rețea îi corespund comunicația WiFi și BLE; iar nivelului de aplicație îi corespund backend-ul, microserviciul de inteligență artificială, baza de date și aplicația web, toate găzduite pe infrastructură cloud.

4.1.1 Separarea în straturi a aplicației software

Partea software a sistemului a fost proiectată conform principiilor unei arhitecturi stratificate (clean architecture), în care responsabilitățile sunt împărțite între mai multe straturi cu dependențe orientate într-un singur sens, dinspre exterior spre interior. Concret, soluția este organizată în următoarele proiecte:

- **Domain:** conține entitățile fundamentale ale sistemului (utilizator, persoană monitorizată, măsurătoare etc.) și interfețele depozitelor de date. Acest strat nu depinde de niciun alt strat și nu cunoaște detalii de implementare.
- **Application:** conține logica de business, organizată în servicii, și interfețele acestora. Depinde exclusiv de stratul Domain.
- **Infrastructure:** conține implementările concrete, anume contextul bazei de date și depozitele realizate cu Entity Framework Core, precum și integrările cu servicii externe. Depinde de Application și Domain.
- **API:** stratul de prezentare al backend-ului, care expune controlerele REST, serviciile de fundal și configurarea aplicației.
- **Client:** aplicația web Blazor WebAssembly, executată în browserul utilizatorului.
- **Shared:** conține obiectele de transfer de date (DTO) folosite în comun de backend și de client, asigurând un contract unic de comunicare.

Avantajul acestei separări este că logica de business rămâne independentă de tehnologiile concrete: baza de date SQLite, de exemplu, ar putea fi înlocuită fără a modifica straturile Domain și Application. Totodată, structura facilitează testarea izolată a fiecărui strat și întreținerea pe termen lung.

4.1.2 Microserviciul de inteligență artificială

Spre deosebire de celelalte componente software, scrise în C#, componenta de inteligență artificială este un microserviciu independent, scris în Python. Această decizie de proiectare a fost luată deliberat, din două motive: ecosistemul Python oferă cele mai mature biblioteci de învățare automată, iar izolarea modelului într-un serviciu propriu permite reantrenarea sau înlocuirea acestuia fără a afecta restul aplicației. Backend-ul comunică cu microserviciul AI prin

cereri HTTP, cu un timp maxim de așteptare scurt, astfel încât o eventuală indisponibilitate a serviciului AI să nu blocheze fluxul principal.

4.1.3 Comunicarea între componente

Componentele sistemului comunică prin trei mecanisme distincte. Dispozitivul IoT și aplicația web comunică cu backend-ul prin **API-ul REST**, peste HTTPS. Backend-ul comunică cu microserviciul AI tot prin HTTP. Pentru actualizarea instantanee a interfeței și pentru livrarea notificărilor push, backend-ul comunică cu aplicația web printr-o conexiune permanentă **WebSocket**, gestionată de biblioteca SignalR. Cele trei componente principale, adică microserviciul AI, backend-ul și frontend-ul, sunt împachetate în containere Docker și orchestrate prin Docker Compose, ceea ce asigură consistența între mediul de dezvoltare și cel de producție. Ansamblul componentelor și al legăturilor dintre ele este prezentat în figura 5.

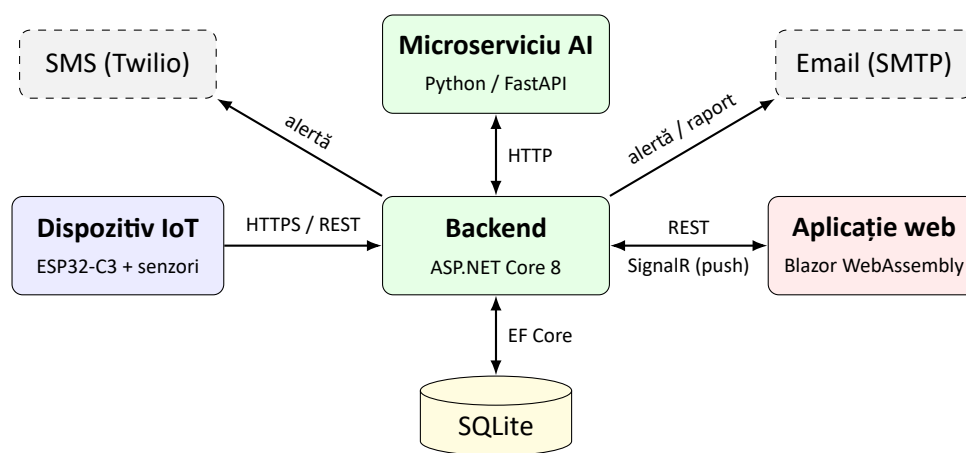


Figura 5: Arhitectura generală a sistemului LifeAlertPlus, cu backend-ul în centru și serviciile externe (Twilio, SMTP) reprezentate prin contururi punctate.

4.1.4 Modelul de deployment

Cele trei componente software ale sistemului, anume microserviciul AI, backend-ul și aplicația web, sunt împachetate fiecare într-o imagine Docker proprie și pornite coordonat. Această izolare a fost o decizie de proiectare deliberată: fiecare componentă poate fi actualizată, repornită sau scalată independent de celelalte, iar mediul de execuție rămâne identic în dezvoltare și în producție. Datele persistente, adică baza de date SQLite, sunt păstrate într-un volum separat de containere, astfel încât informația să nu se piardă la reconstruirea sau actualizarea acestora.

În producție, sistemul este găzduit pe infrastructura cloud Microsoft Azure. Procesul de construire și publicare a imaginilor este automatizat printr-un flux de integrare și livrare continuă (CI/CD): la fiecare modificare a codului, componentele sunt reconstruite și, după validare, publicate automat, ceea ce reduce riscul erorilor manuale și asigură consistența între versiuni.

4.2 Componentele sistemului

4.2.1 Dispozitivul portabil

Dispozitivul portabil este construit în jurul microcontrollerului ESP32-C3 Mini și integrează senzorii descriși în subcapitolul 2.2: MAX30100 pentru puls și SpO_2 , DS18B20 pentru temperatură, MPU6050 pentru mișcare și NEO-6M pentru poziție. Microcontrollerul colectează periodic datele de la senzori, calculează local mărimi derivate, le afișează pe ecranul OLED și le transmite

către backend. Frecvența de transmitere este configurabilă, fiind stabilită de aparținător din aplicația web. Atunci când conexiunea la internet nu este disponibilă, măsurătorile sunt stocate temporar într-o coadă locală pe dispozitiv și transmise ulterior, la revenirea conexiunii. Dispozitivul trimite periodic și un mesaj de tip *heartbeat*, care raportează starea sa tehnică, anume puterea semnalului, memoria liberă, timpul de funcționare și numărul de măsurători aflate în coada de așteptare.

4.2.2 Backend-ul

Backend-ul reprezintă nucleul logic al sistemului. El expune API-ul REST prin care comunică dispozitivele și aplicația web, validează și autorizează cererile, persistă datele în baza de date și coordonează evaluarea stării de sănătate și declanșarea alertelor. Pe lângă controlerele care răspund cererilor, backend-ul include și servicii de fundal care rulează continuu, independent de cereri: serviciul de monitorizare a alertelor, care analizează măsurătorile primite pentru a detecta situații de risc susținute, și serviciul de raportare zilnică, care generează și trimite periodic rapoarte de sănătate.

4.2.3 Microserviciul de inteligență artificială

Microserviciul AI primește valorile senzorilor și pragurile personalizate ale pacientului și returnează starea de sănătate prezisă (NORMAL, ALERT sau CRITICAL), health score-ul și o explicație textuală a deciziei. Funcționarea sa internă, anume modelul Random Forest combinat cu reguli medicale deterministe, a fost descrisă în subcapitolul 2.3.

4.2.4 Aplicația web

Aplicația web este interfața prin care utilizatorii umani interacționează cu sistemul. Ea deservește mai multe categorii de utilizatori, cu drepturi diferite: aparținătorul, care monitorizează una sau mai multe persoane și configurează pragurile și preferințele de notificare; medicul, care primește acces la datele unui pacient prin invitație; și administratorul, care gestionează sistemul. Aplicația oferă un tablou de bord cu starea curentă a persoanei monitorizate, vizualizarea grafică a istoricului măsurătorilor, configurarea pragurilor de alertă, gestionarea notificărilor și localizarea pe hartă a pacientului în scenarii de alertă.

4.2.5 Baza de date

Persistența datelor este asigurată de o bază de date relațională SQLite, accesată prin ORM-ul Entity Framework Core. Baza de date stochează utilizatorii, persoanele monitorizate, istoricul măsurătorilor, notificările și datele agregate. Structura sa este detaliată în subcapitolul următor.

4.2.6 Serviciile externe

Sistemul integrează mai multe servicii externe: o platformă de comunicație în cloud pentru trimiterea SMS-urilor, un server SMTP pentru email, furnizorul de identitate Google pentru autentificarea prin OAuth și un serviciu de date geografice pentru identificarea celui mai apropiat spital în cazul unei alerte.

Structura claselor principale ale aplicației, anume entități, servicii și controlere, este prezentată în diagrama de clase din figura 6.

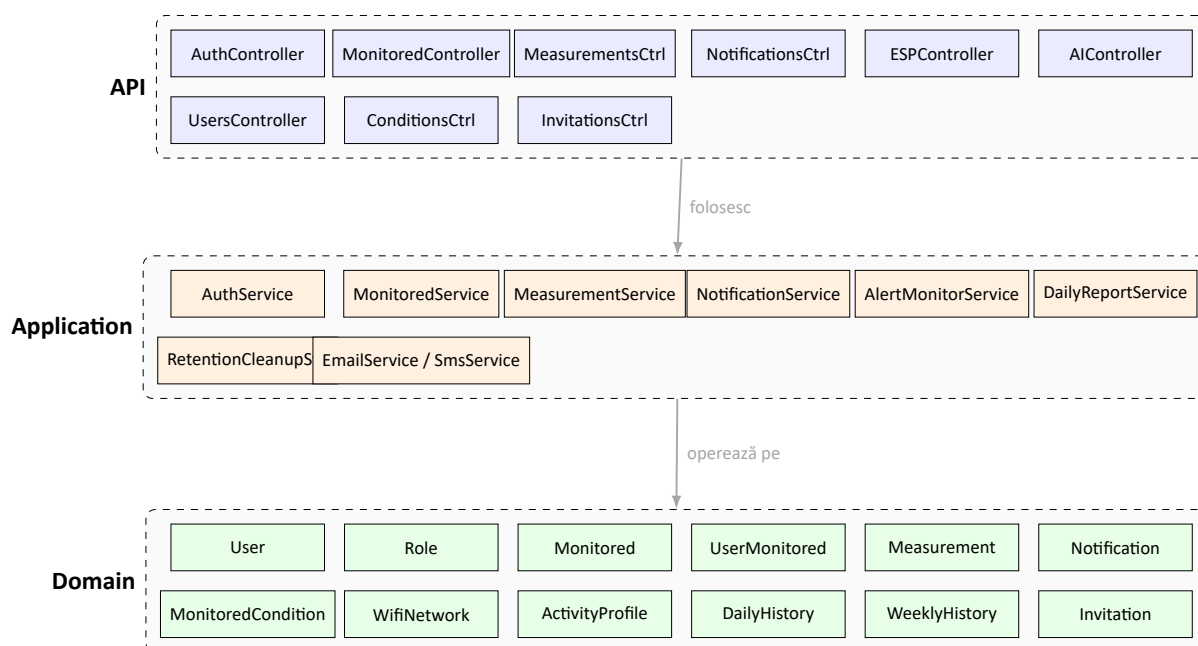


Figura 6: Diagrama de clase simplificată a aplicației LifeAlertPlus, organizată pe trei straturi: API (controlere), Application (servicii) și Domain (entități). Săgețile reprezintă direcția dependențelor între straturi.

4.3 Modelul de date

Modelul de date al sistemului este alcătuit din douăsprezece entități, fiecare corespunzând unei tabele în baza de date. Entitățile principale și rolul lor sunt următoarele:

- **User:** utilizatorul aplicației web (apartinător, medic sau administrator). Conține datele de identificare, credențialele, preferințele de afișare și de notificare (email, push, SMS, raport zilnic), precum și un set de praguri implicite pentru parametrii vitali.
- **Role:** rolul unui utilizator, care îi determină drepturile în aplicație.
- **Monitored:** persoana monitorizată (pacientul vârstnic). Conține datele personale, numărul de serie al dispozitivului asociat, frecvența de actualizare, **pragurile de alertă personalizate** pentru puls, temperatură și SpO₂, precum și **perioada de retenție a datelor** (în zile), după care măsurătorile, notificările și agregările pacientului sunt șterse definitiv.
- **UserMonitored:** tabelă de legătură care modelează relația *mulți-la-mulți* dintre utilizatori și persoanele monitorizate; un utilizator poate monitoriza mai multe persoane, iar o persoană poate fi monitorizată de mai mulți utilizatori (apartinător + medic invitat, de exemplu).
- **Measurement:** o măsurătoare individuală transmisă de dispozitiv, cu valorile de puls, temperatură, SpO₂, activitatea detectată, indicatorul de cădere și coordonatele geografice.
- **MonitoredCondition:** o afecțiune medicală preexistentă a unei persoane monitorizate (hipertensiune, aritmie, diabet etc.), folosită de modulul AI pentru ajustarea evaluării.
- **ActivityProfile:** profilul de activitate al unei persoane, agregat pe ore ale zilei (puls mediu, rata de mișcare, probabilitatea de somn).
- **Notification:** o notificare generată de sistem, asociată unei persoane monitorizate și, opțional, unui utilizator destinatar. Pe lângă conținutul mesajului, înregistrează și **feedback-ul**

utilizatorului cu privire la veridicitatea alertei: dacă alerta a fost rezolvată (vitalele au revenit la normal), notificarea primește un marcaj de solicitare a feedback-ului, iar la următoarea vizită în aplicație utilizatorul răspunde printr-un popup dacă alerta a fost reală sau falsă.

- **WifiNetwork:** o rețea WiFi configurată pentru dispozitivul fizic al unei persoane monitorizate (SSID + parolă). O persoană poate avea până la trei rețele înregistrate (de exemplu rețeaua de acasă și un hotspot mobil), pe care dispozitivul le încearcă la conectare în ordinea înregistrării.
- **DailyHistory și WeeklyHistory:** date agregate zilnic, respectiv săptămânal (valori minime/maxime, medii, numărul de căderi și de alerte), folosite pentru rapoarte și statistici.
- **Invitation:** o invitație trimisă unui medic pentru a-i acorda acces la datele unui pacient, identificată printr-un token unic cu termen de expirare.

4.3.1 Relațiile dintre entități

Relația centrală a modelului este cea *mulți-la-mulți* dintre User și Monitored, realizată prin tabela de legătură UserMonitored, a cărei cheie primară este compusă din cele două chei străine. Fiecare persoană monitorizată este, totodată, în relație *unu-la-mulți* cu măsurătorile, notificările, afecțiunile, profilele de activitate și înregistrările de istoric care îi aparțin. Ștergerea unei persoane monitorizate determină ștergerea în cascadă a datelor sale dependente, în timp ce relațiile cu utilizatorii sunt protejate la ștergere, pentru a păstra integritatea referențială.

Pentru a optimiza performanța interogărilor frecvente, au fost definite indexuri pe cheile străine cele mai utilizate, în special pe identificatorul persoanei monitorizate din tabelele de măsurători și notificări. Token-ul fiecărei invitații este indexat unic, iar combinația dintre o persoană monitorizată și o afecțiune este, de asemenea, unică, pentru a împiedica înregistrările duplicate.

4.3.2 Ștergerea datelor

Sistemul folosește două politici complementare de ștergere, în funcție de tipul datelor. Pentru entitățile cu rol identitar, anume User și Monitored, se aplică **ștergere logică** (soft delete): înregistrările primesc o dată în câmpul DeletedAt și sunt excluse din interogările obișnuite, păstrând astfel integritatea istoricului medical și permițând recuperarea în caz de eroare. Pentru entitățile de tip volumetric, anume Measurement, Notification, DailyHistory și WeeklyHistory, se aplică **ștergere fizică** (hard delete) prin *job-ul* zilnic de retenție descris în subcapitolul 4.4: înregistrările mai vechi decât perioada configurată (DataRetentionDays de pe persoana monitorizată) sunt eliminate definitiv, atât pentru a respecta principiul minimizării datelor cerut de regulamentul GDPR, cât și pentru a limita creșterea bazei de date sub fluxul continuu de măsurători de la dispozitiv.

4.3.3 Datele agregate și istoricul

Pe lângă entitățile care reflectă starea curentă, modelul include entități de **date agregate**, calculate periodic din măsurătorile brute. DailyHistory reține, pentru fiecare zi, valorile minime și maxime ale pulsului și temperaturii, numărul de căderi și numărul de alerte; WeeklyHistory reține medii și totaluri la nivel de săptămână. Aceste entități nu adaugă informație nouă, ci o sintetizează: ele permit afișarea rapidă a statisticilor și generarea rapoartelor fără a parcurge de fiecare dată întregul istoric de măsurători, care crește considerabil în timp. O a treia formă de agregare este ActivityProfile, care reține distribuția activității pacientului pe orele zilei și stă la baza profilului de activitate zilnică.

Diagrama entitate-relație a modelului de date este prezentată în figura 7.

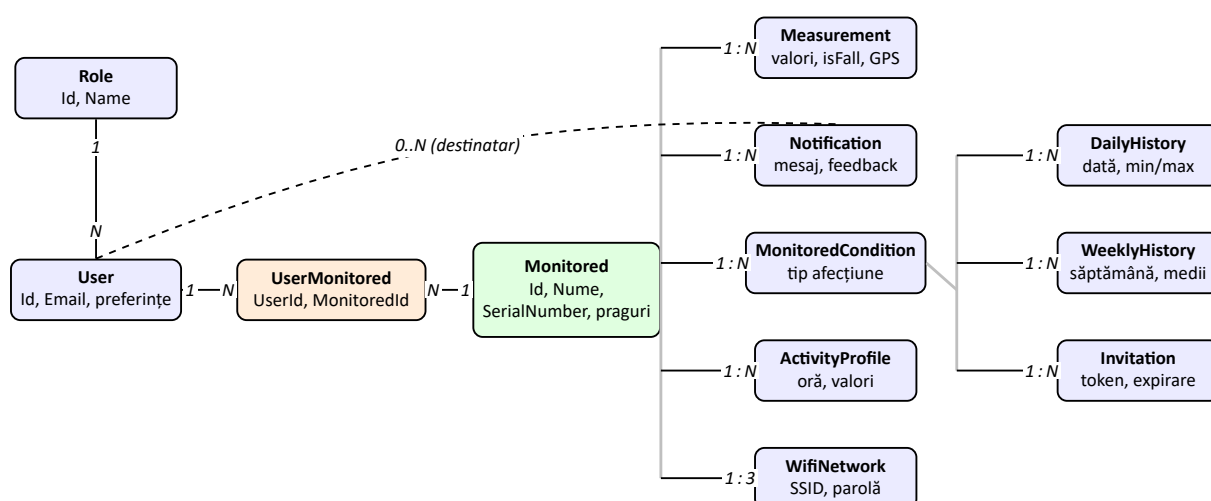


Figura 7: Diagrama entitate-relație a bazei de date LifeAlertPlus. Relația centrală *mulți-la-mulți* dintre User și Monitored se realizează prin tabela de legătură UserMonitored. Toate entitățile dependente sunt legate *unu-la-mulți* de Monitored, iar relația punctată indică o legătură opțională (un destinatar pentru o notificare).

4.4 Fluxuri de funcționare

4.4.1 Achiziția și transmiterea unei măsurători

Fluxul principal al sistemului începe pe dispozitivul portabil. La intervalul de timp configurat, microcontrollerul citește valorile tuturor senzorilor, filtrează citirile evident eronate și calculează mărimile derivate (pulsul, magnitudinea accelerației, etc.). Pachetul de date rezultat, care include și un **indicator boolean** setat în cazul în care detectorul de căderi a confirmat un eveniment de cădere, este transmis backend-ului printr-o cerere HTTP către endpoint-ul de ingestie, autentificată printr-o cheie de dispozitiv transmisă în antetul cererii. Dacă transmiterea eșuează din cauza lipsei conexiunii, măsurătoarea este adăugată în coada locală a dispozitivului și retransmisă ulterior. La recepție, backend-ul validează cheia de dispozitiv, asociază măsurătoarea persoanei monitorizate pe baza numărului de serie, o persistă în baza de date și o transmite mai departe serviciului de monitorizare a alertelor împreună cu indicatorul de cădere; o cădere raportată de dispozitiv ridică automat severitatea evaluării la nivel critic, indiferent de valorile parametrilor vitali.

4.4.2 Evaluarea stării prin modulul AI

Fiecare măsurătoare primită este trimisă microserviciului de inteligență artificială, împreună cu pragurile personalizate ale pacientului și lista afecțiunilor sale. Microserviciul returnează starea prezisă, health score-ul și explicația. Pentru a evita ca o indisponibilitate a serviciului AI să blocheze fluxul, cererea are un timp de așteptare scurt, iar în lipsa unui răspuns sistemul recurge la evaluarea pe bază de reguli.

4.4.3 Calculul health score-ului

Health score-ul este un indice numeric între 0 și 100, în care valoarea 0 corespunde unei stări complet stabile, iar valori ridicate corespund unui risc crescut. Spre deosebire de o alertă bazată pe depășirea unui singur prag, health score-ul este o **sumă ponderată de contribuții**, fiecare parametru vital adăugând la scor o cantitate proporțională cu severitatea abaterii sale.

Pragurile utilizate sunt cele personalizate ale pacientului, cu valori clinice implicite atunci când acestea nu au fost configurate.

Contribuțiile sunt agregate astfel:

- **Saturația oxigenului** are ponderea cea mai mare: o valoare sub pragul critic adaugă o contribuție majoră, iar o valoare situată între pragul critic și cel minim adaugă o contribuție proporțională cu distanța față de prag.
- **Frecvența cardiacă** adaugă o contribuție mare la depășirea pragului maxim sau la coborârea sub cel minim și o contribuție moderată în zonele de avertizare aflate înaintea acestor praguri.
- **Temperatura corporală** contribuie similar, atât pentru valori febrile, cât și pentru hipotermie.
- **Imobilitatea**, manifestată prin accelerație foarte redusă, care poate indica o cădere urmată de imobilizare, adaugă o contribuție suplimentară.
- **Afecțiunile preexistente** aplică modificatori specifici: de exemplu, în cazul unei aritmii, un puls anormal adaugă o penalizare suplimentară, întrucât are o semnificație clinică mai gravă pentru acel pacient.

Ponderile concrete utilizate în agregare sunt sintetizate în tabelul 2. Pragurile la care se raportează condițiile sunt cele personalizate ale pacientului; în lipsa configurării, se folosesc valori clinice implicite (frecvență cardiacă 50–130 bpm, temperatură 35–39 °C, saturație minimă 95%).

Parametru	Condiție	Contribuție
<i>Parametri vitali</i>		
SpO ₂	Sub pragul critic de saturație	+60
SpO ₂	Între pragul critic și pragul minim	+30 (gradual)
Frecvență cardiacă	Peste pragul maxim	+25
Frecvență cardiacă	Zona de avertizare înaltă (peste 85% din maxim)	+12
Frecvență cardiacă	Sub pragul minim	+20
Frecvență cardiacă	Zona de avertizare joasă (sub 120% din minim)	+10
Temperatură corporală	Peste pragul maxim	+20
Temperatură corporală	Zona de avertizare înaltă (cu 1 °C sub maxim)	+10
Temperatură corporală	Sub pragul minim	+20
Temperatură corporală	Zona de avertizare joasă (cu 1 °C peste minim)	+10
Imobilitate	Accelerație sub 0,2 g (posibilă cădere/imobilizare)	+8
<i>Modificatori în funcție de afecțiunile preexistente</i>		
Hipertensiune	Puls peste 100 bpm	+8
Aritmie	Puls peste 110 bpm sau sub 52 bpm	+10
Risc de infarct	Puls peste 110 bpm și temperatură peste 37,8 °C	+12
Diabet	Temperatură peste 38 °C	+8
Parkinson / epilepsie	Accelerație sub 0,5 g	+5

Tabela 2: Ponderile contribuțiilor în calculul health score-ului.

Scorul rezultat este limitat la intervalul [0, 100]. Avantajul acestui mecanism este că surprinde **corelația dintre mai mulți parametri**: mai multe abateri ușoare, fiecare insuficientă pentru a declanșa singură o alertă, se pot cumula într-un scor ridicat, semnalând o degradare globală a stării pacientului care altfel ar trece neobservată.

4.4.4 Predicția de tendințe

Pe lângă evaluarea stării instantanee, sistemul oferă o **predicție de tendințe pe termen scurt**, menită să anticipeze atingerea unui prag clinic înainte ca acesta să fie efectiv depășit. Serviciul de monitorizare păstrează, pentru fiecare pacient, un buffer cu măsurătorile din ultimele două minute și calculează, pentru fiecare parametru vital (puls, temperatură, SpO₂), media și **panta** (rata de variație) pe acest interval. Atunci când panta indică o evoluție constantă către o valoare critică, sistemul estimează timpul rămas până la atingerea pragului prin extrapolare liniară și produce o etichetă explicativă.

De exemplu, o temperatură în creștere constantă generează predicția *Posibilă febră* sau *Febră*

în evoluție, împreună cu numărul aproximativ de secunde până la pragul de 38,5 °C; analog, se anticipează tahicardia, bradicardia sau hipoxia. Aceste predicții sunt afișate în interfața de vizualizare a pacientului (figura 8) ca avertismente gradate (atenționare sau pericol) și au rol pur informativ pentru aparținător, fără a declanșa singure notificări, întrucât se bazează pe extrapolare și nu pe o depășire efectivă a pragului. Spre deosebire de health score, care reflectă starea curentă, predicția de tendințe privește în viitorul apropiat (de ordinul minutelor).

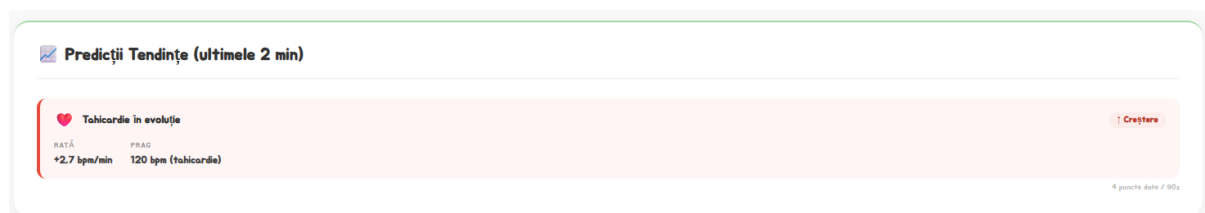


Figura 8: Predicțiile de tendințe afișate în pagina pacientului: etichetă, valoare curentă, rată de variație și timp estimat până la prag.

4.4.5 Declanșarea și escaladarea unei alerte

Detectarea unei situații de risc nu declanșează imediat o notificare. Serviciul de monitorizare a alertelor menține, pentru fiecare persoană monitorizată, un buffer cu măsurătorile recente și aplică un **prag de persistență**: o stare anormală trebuie să se mențină o durată minimă înainte de a genera o alertă, eliminând astfel valorile tranzitorii cauzate de un contact imperfect al senzorului. Severitatea de bază a alertei este apoi ajustată de motorul de reguli pe baza afecțiunilor pacientului, care poate ridica nivelul de severitate și poate adăuga recomandări specifice.

Odată confirmată, alerta este livrată simultan prin toate canalele activate în preferințele aparținătorului, anume notificare push, email și/sau SMS. Pragul de persistență este de aproximativ două minute în cazul stărilor critice; după ce este atins, toate canalele alese de utilizator primesc notificarea în același moment, fără ca SMS-ul să fie întârziat suplimentar față de celelalte. Atâta vreme cât starea critică persistă, sistemul retrimite notificarea la fiecare două minute; pentru alertele non-critice, un mecanism de **cooldown** împiedică retransmiterea aceleiași alerte pentru aceeași persoană într-un interval scurt, evitând inundarea destinatarului cu mesaje. În cazul alertelor critice, sistemul include în notificare și informații despre cel mai apropiat spital față de poziția pacientului. Întregul flux, de la achiziția măsurătorii până la livrarea alertei, este ilustrat în figura 9.

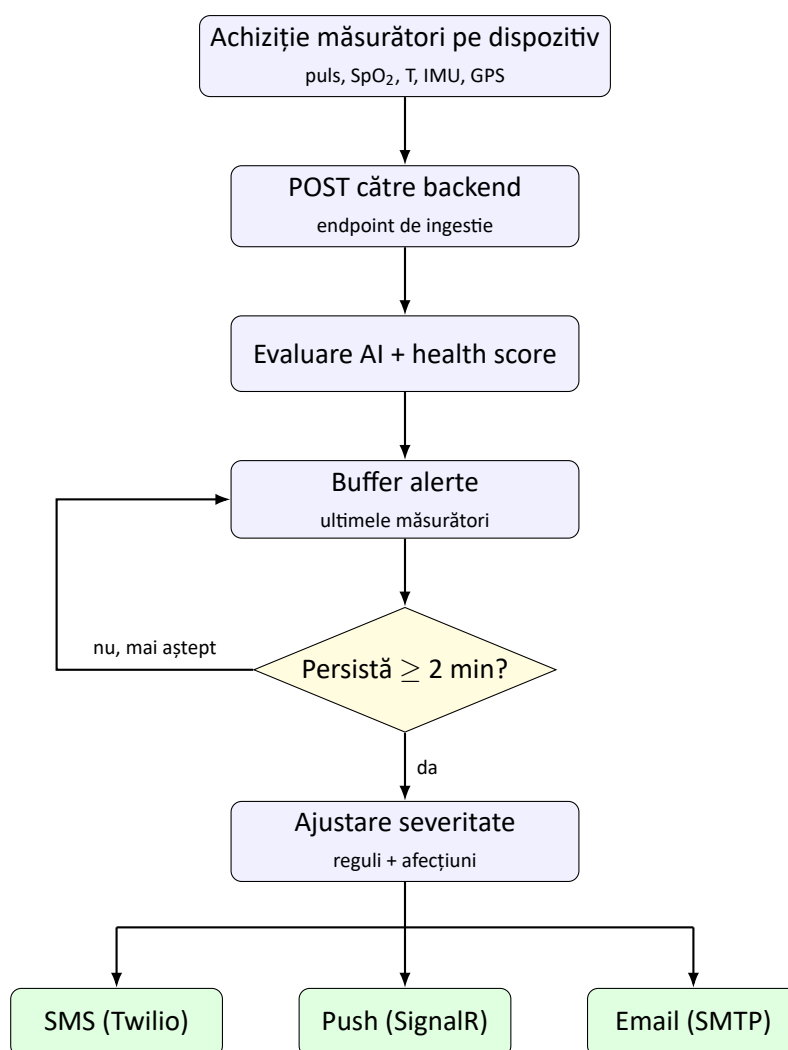


Figura 9: Fluxul de declanșare și escaladare a unei alerte.

4.4.6 Butonul de panică

Independent de fluxul automat, pacientul poate declanșa el însuși o alertă apăsând butonul de panică al dispozitivului. Această acțiune trimite o cerere dedicată către backend, care generează imediat o alertă către aparținător, fără a aștepta pragul de persistență și indiferent de valorile senzorilor, întrucât reprezintă o solicitare explicită de ajutor. Mesajul livrat aparținătorului specifică explicit faptul că *vârstnicul nu se simte în siguranță* și a apăsă butonul de panică, iar livrarea se face prin **toate canalele activate în preferințele aparținătorului** (notificare push, email și SMS), pentru a maximiza probabilitatea de a fi observat imediat. Dacă dispozitivul are un fix GPS valid în momentul apăsării, notificarea include și ultima poziție cunoscută a pacientului, împreună cu informații despre cel mai apropiat spital de urgență. În caz de pierdere temporară a conexiunii, alerta de panică este salvată într-o coadă locală separată de cea a măsurătorilor obișnuite și are prioritate la retransmitere când conexiunea revine.

4.4.7 Detecția căderilor pe dispozitiv

Detecția căderilor este realizată **integral pe dispozitiv**, ca task separat care eșantionează accelerometrul MPU6050 la o frecvență de aproximativ 50 Hz. Algoritmul, bazat pe trecerea accelerației prin trei faze succesive, este implementat ca o mașină de stări cu trei stări active: *IDLE*, *IN_FREEFALL* și *MONITORING_STILLNESS*. Pentru fiecare eșantion se calculează magnitudinea

vectorului de accelerație, $|\vec{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2}$, și se aplică tranzițiile prezentate în figura 10.

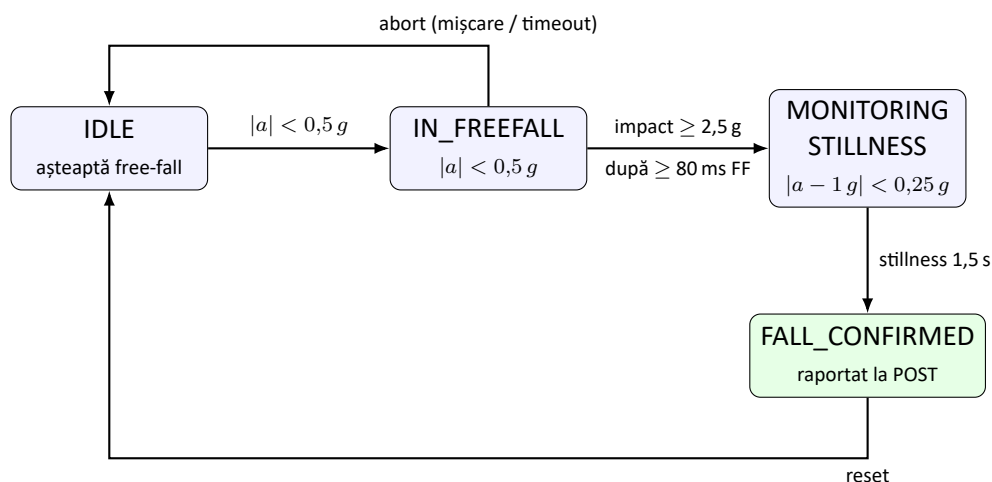


Figura 10: Mașina de stări a detectorului de căderi rulat pe dispozitiv.

Pragurile au fost alese empiric pe baza literaturii de specialitate pentru detectori de tip prag: free-fall sub 0,5 g susținut minim 80 ms, impact peste 2,5 g într-o fereastră de cel mult 800 ms după free-fall, urmat de o perioadă de stillness de 1,5 s în care magnitudinea accelerației rămâne în jurul valorii de 1 g (gravitația), cu variații sub 0,25 g. La confirmare, task-ul setează un flag pe care bucla principală îl include în următoarea cerere către endpoint-ul de ingestie. Fluxul restant se identifică cu cel al unei măsurători obișnuite, însă serverul ridică automat severitatea la nivel critic atunci când indicatorul de cădere este activ, declanșând livrarea pe toate canalele activate.

4.4.8 Profilul de activitate și detecția anomaliilor comportamentale

Pe lângă măsurătorile instantanee, sistemul construiește pentru fiecare pacient un **profil de activitate zilnică** agregat pe orele zilei dintr-o fereastră glisantă de 14 zile. Sursa de date este eticheta de activitate (moving / stationary) calculată în firmware pentru fiecare măsurătoare (procedeu descris în subcapitolul de implementare a firmware-ului). Pentru fiecare oră a zilei, serviciul de profilare păstrează trei valori derivate: pulsul mediu, **rata de mișcare** (proportia măsurărilor etichetate moving) și **probabilitatea de somn** (proportia măsurărilor etichetate stationary la care pulsul este sub 75 bpm). Profilul este recalculat în fundal o singură dată per pacient, atunci când e folosit prima oară, și păstrat într-o memorie locală cu expirare la 4 ore pentru a evita interogări repetate pe baza de date.

Pe baza acestui profil, la fiecare măsurătoare nouă serviciul de monitorizare verifică dacă starea curentă se abate semnificativ de la tiparul obișnuit al pacientului la ora respectivă, declanșând una dintre două **anomalii comportamentale**:

- **Inactivitate prelungită:** în ultimele 15 minute toate măsurătorile au fost etichetate stationary, deși la ora curentă pacientul se mișcă în mod obișnuit cel puțin 65% din timp (după profilul de 14 zile). Acest scenariu poate semnaliza o situație de risc (cădere nedetectată, leșin, imobilizare).
- **Activitate nocturnă:** măsurătoarea curentă este etichetată moving într-o oră în care pacientul doarme de obicei cel puțin 70% din timp. Acest scenariu poate semnaliza agitație nocturnă, ridicare neașteptată sau o stare alterată.

Atât profilul cât și detecția anomaliilor sunt activate numai dacă există suficiente date istorice pentru ora respectivă (minim 20 de măsurători); pentru pacienți nou înregistrați mecanismul rămâne inactiv până se acumulează suficiente date pentru un profil semnificativ statistic. Notificările comportamentale sunt limitate prin același mecanism de *cooldown* (minim 30 de minute între două notificări de același tip pentru același pacient) ca să evite inundarea aparținătorului.

4.4.9 Configurarea rețelelor WiFi de la distanță

Pentru a permite folosirea dispozitivului în mai multe medii (de exemplu, acasă și pe un hotspot mobil), aparținătorul poate înregistra din aplicația web până la trei rețele WiFi asociate dispozitivului unui pacient. Lista este persistată în baza de date; dispozitivul interoghează periodic backend-ul pentru ultima configurație disponibilă, în paralel cu *heartbeat-ul* obișnuit. Atunci când configurația citită diferă de cea salvată local pe dispozitiv, lista este rescrisă în memoria nevolatilă (evitând astfel uzura memoriei flash), iar rețelele noi devin active la următoarea repornire a dispozitivului, fără a întrerupe sesiunea curentă. Fluxul complet este ilustrat în figura 11.

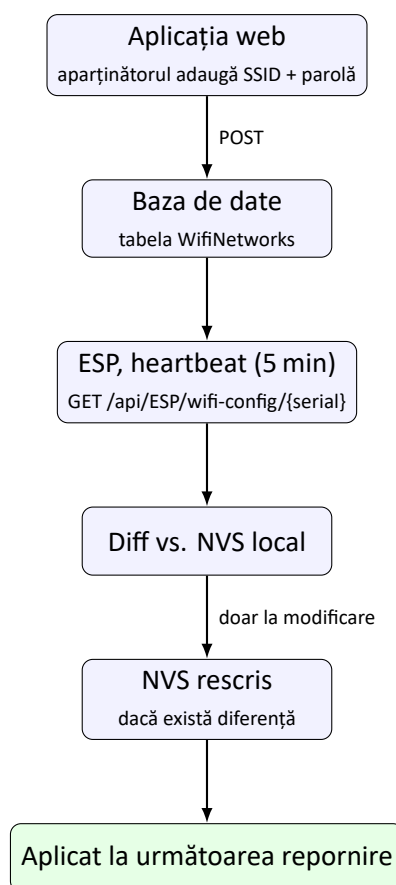


Figura 11: Fluxul de sincronizare a configurației WiFi între aplicația web și dispozitivul fizic.

4.4.10 Feedback pentru alarme false

Pentru a permite ajustarea pragurilor și pentru a oferi date concrete despre rata de alerte false, sistemul implementează o **buclă de feedback** cu utilizatorul aparținător. Atunci când o stare de alertă (puls anormal, temperatură ridicată, SpO₂ scăzut etc.) se rezolvă, adică valorile pacientului revin la normal după ce o notificare a fost deja trimisă, ultima notificare a episodului este marcată automat ca *așteptând feedback*. La următoarea vizită a aparținătorului în apli-

cație, un popup neintruziv afișează informațiile alertei (pacient, momentul evenimentului) și întreabă explicit dacă alerta a fost reală sau o alarmă falsă. Răspunsul este înregistrat pe notificare și permite raportarea ulterioară a ratei reale de alerte false, calculată din datele etichetate de utilizator. Utilizatorul poate amâna răspunsul, caz în care popup-ul re apare la următoarea sesiune.

4.4.11 Politica de retenție a datelor

Aplicația implementează o politică de retenție configurabilă **per pacient**. Pentru fiecare persoană monitorizată, aparținătorul poate alege durata maximă de păstrare a datelor (30, 90, 180 sau 365 de zile) din modalul de editare al pacientului. Un *job* de fundal rulează automat o dată pe zi, la miezul nopții (ora locală configurată în setările serverului), și șterge definitiv din baza de date toate măsurătorile, notificările și agregările zilnice / săptămânale mai vechi decât perioada configurată. Spre deosebire de ștergerea logică folosită pentru entitățile cu rol identitar, această curățare este o ștergere fizică ireversibilă, motivată de respectarea principiului de minimizare a datelor și de limitarea creșterii bazei de date sub fluxul continuu de măsurători de la dispozitiv.

4.4.12 Sincronizarea datelor stocate local

Atunci când dispozitivul funcționează fără conexiune la internet, măsurătorile se acumulează în coada locală. La revenirea conexiunii, dispozitivul transmite măsurătorile aflate în așteptare, în ordinea înregistrării lor. Astfel, istoricul medical rămâne complet chiar și în condiții de conectivitate intermitentă, în concordanță cu cerința de fiabilitate a sistemelor IoMT prezentată în subcapitolul 2.1.

4.4.13 Înrolarea utilizatorilor și asocierea dispozitivului

Înainte de a putea monitoriza o persoană, un aparținător își creează un cont și îl confirmă prin email. Ulterior, el adaugă o persoană monitorizată, completând datele acesteia și asociind numărul de serie al dispozitivului fizic. Această asociere este cheia pe baza căreia backend-ul leagă, ulterior, fiecare măsurătoare primită de la dispozitiv de persoana corectă. Tot la acest moment sunt configurate pragurile de alertă personalizate și afecțiunile preexistente, care vor fi folosite la evaluarea stării de sănătate.

4.4.14 Invitarea medicului și exportul datelor

Un aparținător poate acorda unui medic acces la datele unei persoane monitorizate. Sistemul generează în acest scop o invitație identificată printr-un token unic, cu termen de expirare, transmisă medicului prin email; la acceptarea ei, medicul primește drept de vizualizare asupra datelor pacientului respectiv. În plus, istoricul de măsurători poate fi exportat în mod structurat și transmis medicului curant, pentru un interval de timp selectabil, facilitând consultarea datelor în afara aplicației.

5 Implementarea aplicației

Acest capitol detaliază implementarea efectivă a fiecărei componente proiectate în capitolul anterior. Sunt prezentate, pe rând, hardware-ul dispozitivului portabil, firmware-ul care îl operează, backend-ul și API-ul REST, microserviciul de inteligență artificială, aplicația web și sistemul de notificări multi-canal.

5.1 Hardware-ul dispozitivului portabil

Dincolo de descrierea conceptuală a senzorilor din subcapitolul 2.2, realizarea efectivă a dispozitivului a presupus o serie de decizii practice de proiectare hardware: alegerea topologiei magistralei de comunicație, dimensionarea circuitului de alimentare, alocarea atentă a pinilor microcontrollerului și implementarea unei proceduri de auto-testare care să facă vizibile defectele de montaj încă de la pornire. Aceste aspecte sunt detaliate în continuare.

5.1.1 Schema de conexiuni și maparea pinilor GPIO

Microcontrollerul ESP32-C3 Mini dispune de un număr limitat de pini GPIO de uz general, iar unii dintre aceștia au funcții speciale (pini de *strapping* care influențează modul de boot, pini rezervați USB-JTAG-ului nativ, pini de alimentare). Maparea folosită în acest proiect, sintetizată în tabelul 3, a fost stabilită astfel încât funcțiile care necesită pull-up extern sau întreruperi pe muchie să fie alocate pe pini neutri, iar pinii rezervați USB să rămână disponibili pentru depanare.

GPIO	Componentă	Observații
0	DS18B20 (1-Wire)	Pull-up extern 4,7 k Ω ; mod open-drain
1	GPS NEO-6M (TX → ESP RX)	UART1 RX, 9600 baud, NMEA
2	GPS NEO-6M (RX ← ESP TX)	UART1 TX
3	Buton panică	Pull-up intern; întrerupere pe muchie descendentă
4	Buton pornire/oprire	Pull-up intern; întrerupere pe muchie descendentă
5	LED galben (eroare / offline / sleep)	Active-HIGH
6	LED verde (sistem activ)	Active-HIGH
8	I ² C SDA (TCA9548)	Pull-up intern activat
9	I ² C SCL (TCA9548)	Pull-up intern activat

Tabela 3: Maparea pinilor GPIO ai microcontrollerului ESP32-C3 Mini.

[Spațiu pentru figură: schema electrică a dispozitivului, prezentând ESP32-C3 Mini, multiplexorul TCA9548A, senzorii MAX30100/MPU6050/DS18B20/NEO-6M, ecranul OLED, butoanele și LED-urile, cu pinii GPIO corespunzători și rezistoarele de pull-up evidențiate.]

Figura 12: Schema de conexiuni a dispozitivului portabil.

5.1.2 Magistrala I²C și multiplexarea TCA9548A

Trei dintre componentele dispozitivului, anume ecranul OLED SSD1306, senzorul MAX30100 și senzorul MPU6050, comunică prin magistrala I²C. Întrucât MAX30100 are o adresă fixă (0x57), iar OLED-ul și MPU6050 au adrese predefinite care nu pot fi modificate fără remontare hardware, conectarea directă pe aceeași magistrală ar genera, în practică, comportament instabil din cauza încărcării capacitive cumulate și a interferențelor între dispozitive cu cerințe diferite de *timing*.

Soluția adoptată este utilizarea unui multiplexor I²C **TCA9548A** pe 8 canale, care expune fiecărui dispozitiv un canal izolat electric. Microcontrollerul selectează canalul activ printr-o scriere unică pe registrul multiplexorului (adresa 0x70), efectuează tranzacția I²C cu senzorul vizat, apoi închide canalul. Această secvență *open/read/close* este aplicată sistematic în firmware pentru a preveni conflictele de adresă atunci când mai mulți senzori partajează aceleași numere de adresă pe magistrale diferite. Canalul 0 a fost alocat senzorului MAX30100, canalul 1 ecranului OLED, iar canalul 2 senzorului MPU6050.

5.1.3 Sistemul de alimentare

Alimentarea dispozitivului portabil este asigurată de un acumulator **LiPo 3,7 V / 1500 mAh** (model 103048, AM Electronics), ales pentru raportul favorabil între capacitate și gabarit (10 × 30 × 48 mm) și pentru protecția integrată împotriva supraîncărcării, supradescărcării și scurtcircuitului. Gestionarea încărcării prin USB-C este realizată de modulul **TP4056**, un circuit integrat dedicat încărcării bateriilor Li-Ion / Li-Po cu un curent maxim de 1 A și protecție completă a celulei.

Întrucât tensiunea nominală a acumulatorului LiPo (3,0–4,2 V) este insuficientă pentru alimentarea simultană a tuturor componentelor în regimul lor optim, sistemul include un convertor **boost MT3608**, capabil să furnizeze până la 28 V / 2 A și configurat în acest caz pentru tensiunea de operare a sistemului (5 V).

Modulul ESP32-C3 Mini integrează un **Brownout Detector (BOD)** care resetează cipul atunci când tensiunea de alimentare coboară sub un prag configurabil (implicit 2,51 V pe ESP32-C3). În timpul testărilor s-a observat că tranzițiile rapide de regim, de exemplu comutarea în modul sleep cu oprirea simultană a LED-urilor senzorului MAX30100 și schimbarea modului de *power save* al radio-ului WiFi, generează pe sursa de alimentare un *transient* suficient de mare încât să declanșeze BOD-ul. Cauza principală o reprezintă curentul-vârf consumat de transceiver-ul WiFi în timpul emisiei, care poate atinge 350 mA, valoare la limita capacității unui cablu USB de calitate medie.

Pentru atenuarea acestui efect, în firmware s-au introdus pauze scurte (50 ms) între acțiunile cu impact electric major, astfel încât condensatoarele de filtraj să aibă timp să-și revină între evenimente. La nivel hardware, soluția recomandată este adăugarea unei perechi de condensatoare de decuplare (100 μ F electrolitic în paralel cu 100 nF ceramic), montate cât mai aproape de pinul de alimentare al microcontrollerului, precum și folosirea unui cablu USB cu fire de alimentare cu rezistență scăzută.

5.1.4 Interfața fizică: LED-uri și butoane

Dispozitivul include două elemente de feedback vizual și două elemente de interacțiune fizică. Cele două LED-uri (verde pentru funcționare normală și galben pentru eroare, lipsă de rețea sau mod sleep) sunt conectate active-HIGH la pinii GPIO 6, respectiv GPIO 5, prin rezistoare de limitare a curentului dimensionate la aproximativ 10 mA per LED.

Cele două butoane momentane sunt conectate între pinul GPIO și masă, iar starea de inactivitate (HIGH) este menținută prin pull-up-urile interne ale microcontrollerului, configurate software. Detecția apăsării se face pe **muchie descendentă** (*negative edge*), printr-o întrerupere GPIO, cu *debounce* software de 50 ms aplicat în handler. Pentru a crește fiabilitatea, firmware-ul implementează în paralel și o cale de *polling*: dacă, dintr-un motiv oarecare, întreruperea nu se declanșează, citirea ciclică a nivelului pinului prinde tot tranziția. Această abordare hibridă a fost utilă în depanare, întrucât a permis distingerea unei probleme de cablaj (pin niciodată LOW la apăsare) de o problemă strict software (pin oscilează corect, dar întreruperea nu se înregistrează).

5.1.5 Diagnoza la pornire și auto-testarea

Pentru ca defectele de montaj să poată fi identificate fără instrumente externe, firmware-ul execută la pornire o procedură de auto-testare în două etape, ale cărei rezultate sunt afișate pe portul serial. Prima etapă, denumită *hardware self-test*, verifică elementele active de interacțiune: face LED-ul verde să clipească de mai multe ori (absența clipirii indică imediat o defecțiune a LED-ului, a rezistorului sau a cablajului) și monitorizează timp de 10 s nivelul pinilor celor două butoane, raportând câte tranziții HIGH→LOW a detectat pe fiecare. Această ultimă măsurare permite distingerea unui buton blocat (pin niciodată HIGH) de un buton deconectat sau cu firul de masă întrerupt (pin niciodată LOW).

A doua etapă, denumită *sensor functional test*, depășește simpla verificare a prezenței pe magistrală (*I²C probe*) și efectuează o citire reală de la fiecare senzor: extrage un eșantion de la MPU6050 și îl validează împotriva valorilor *stuck zero* sau *bus railed* (toți biții la 0 sau la 1, semn al unei defecțiuni de comunicație); trezește senzorul MAX30100 și citește FIFO-ul după un interval suficient pentru acumularea câtorva eșantioane; declanșează o conversie completă a senzorului DS18B20 și verifică integritatea datelor prin codul de control CRC8; verifică prezența caracterelor NMEA pe portul UART al modului GPS. Pentru fiecare senzor se afișează verdictul (FUNCTIONAL, FAULT sau NOT DETECTED) alături de valorile efective citite, ceea ce face vizibile deopotrivă defectele de cablaj, contactul slab al senzorului și erorile de configurare.

Pentru cazul particular al unui buton blocat la masă (pin permanent LOW), firmware-ul include și un *guard* care, după 500 ms de citire continuă, dezactivează butonul respectiv, împiedicând astfel generarea unei avalanșe de evenimente false și păstrând restul sistemului funcțional până la remedierea defecțiunii hardware.

Firmware-ul dispozitivului portabil a fost dezvoltat în limbajul C++, utilizând framework-ul **ESP-IDF** (Espressif IoT Development Framework), mediul de dezvoltare oficial pentru microcontrolerele Espressif. Programul rulează peste sistemul de operare în timp real **FreeRTOS**, care per-

mite organizarea funcționalității în sarcini (*tasks*) concurente și sincronizarea acestora prin grupe de evenimente; de exemplu, transmiterea datelor către server este declanșată doar după ce sarcina de conectare la rețea semnalează stabilirea conexiunii WiFi.

Pornind de la maparea pinilor și de la organizarea hardware descrise în subcapitolul 5.1, firmware-ul definește o abstractizare uniformă pentru fiecare senzor: o funcție de inițializare, o funcție de citire și, după caz, funcții de *wake / sleep* pentru gestionarea consumului. Procedura de auto-testare prezentată anterior este apelată o singură dată, la pornire, după care execuția intră într-o buclă principală responsabilă cu achiziția periodică și transmiterea datelor.

5.1.6 Citirea senzorilor și multiplexarea I2C

Senzorii sunt interogați ciclic, într-o buclă cu perioada de ordinul a câteva sute de milisecunde. Întrucât MAX30100, ecranul OLED și MPU6050 partajează aceeași magistrală I2C, accesul la fiecare dintre ele se face prin selectarea canalului corespunzător al multiplexorului TCA9548A înaintea fiecărei citiri. Pentru a evita coliziunile între task-urile FreeRTOS concurente, accesul la magistrala I2C este serializat printr-un mutex recursiv. Senzorul de temperatură DS18B20 comunică separat, prin protocolul 1-Wire, iar modulul GPS NEO-6M prin interfața UART, datele de poziție fiind extrase din mesajele transmise în format NMEA. Valorile brute sunt validate local, prin compararea cu intervalele fizic posibile, iar citirile evident eronate, generate de un contact imperfect al senzorului cu pielea, sunt eliminate înainte de a fi luate în considerare.

5.1.7 Detecția căderilor în firmware

Detectorul de căderi descris conceptual în subcapitolul 4.4 este implementat ca un task FreeRTOS separat (*fall_task*), care rulează la prioritate egală cu task-ul de citire a pulsului. Task-ul eșantionează accelerometrul MPU6050 la 50 Hz, calculează magnitudinea $|\vec{a}|$ în unități de *g* (factorul de scală 16 384 LSB/g pentru intervalul implicit $\pm 2g$) și aplică mașina de stări cu trei stări active prezentată în figura 10. La confirmarea unei căderi, task-ul setează un *flag* global pe care bucla principală îl consumă atomic la următorul ciclu de POST: include valoarea *true* pentru câmpul *isFall* în payload-ul JSON și resetează flag-ul, garantând că o cădere este raportată o singură dată. În *sleep mode* (activat prin apăsarea butonului dedicat), task-ul își păstrează doar starea IDLE și nu mai consumă resurse de I/O.

Pragurile algoritmului (free-fall 0,5 g, impact 2,5 g, stillness 1,5 s) sunt definite ca *define*-uri C, astfel încât să poată fi recalibrate fără modificarea logicii.

5.1.8 Clasificarea activității pe dispozitiv

În paralel cu mașina de stări a detectorului de căderi, același flux de eșantioane MPU6050 alimentează un **acumulator de mișcare** folosit pentru a clasifica activitatea purtătorului între două POST-uri consecutive. Pentru fiecare sample se calculează abaterea magnitudinii față de gravitație, $\delta = ||\vec{a}| - 1 g|$, iar valorile sunt agregate într-o structură protejată de mutex: numărul de sample-uri, suma cumulată a abaterilor și valoarea maximă observată în fereastră. La fiecare ciclu de POST (5 s, 250 sample-uri la 50 Hz), funcția care construiește payload-ul JSON face un snapshot atomic al acumulatorului, îl resetează și clasifică:

- dacă media abaterii depășește 0,05 g (mers susținut peste zgomotul senzorului de 0,02 g la repaus), sau
- dacă vârful izolat depășește 0,20 g (mișcare scurtă: ridicat în picioare, întors în pat)

atunci eticheta de activitate este "moving"; altfel, "stationary". Sub un minim de 10 sample-uri (MPU oprit, sleep mode), câmpul este omis din JSON, astfel încât backend-ul să trateze

absența lui ca lipsă de date în loc să interpreteze pe baza unei ferestre incomplete. Eticheta este apoi persistată pentru fiecare măsurătoare în baza de date și alimentează profilul de activitate zilnică descris în subcapitolul 4.4.

Avantajul acestei soluții este că reutilizează exact aceleași eșantioane pe care detectorul de căderi le citește oricum la 50 Hz, deci costul suplimentar este aproape zero (o operație de incrementare și o comparație per sample). În același timp, fereastra de 5 s este suficient de lungă pentru a netezi zgomotul senzorului și suficient de scurtă pentru ca acumularea de stări (15 minute de inactivitate într-o oră de obicei activă, de exemplu) să rămână reactivă.

5.1.9 Sincronizarea configurației WiFi

Pe lângă transmiterea datelor, firmware-ul implementează și un mecanism de **sincronizare a configurației WiFi** cu backend-ul, conform fluxului din figura 11. La fiecare *heartbeat* (interval de 5 minute), dispozitivul efectuează o cerere GET către endpoint-ul de configurație, primește lista cu până la trei perechi SSID + parolă și o compară byte-cu-byte cu valorile salvate local în NVS. Dacă există diferență, lista nouă este scrisă în NVS (folosind aceleași chei pe care funcția de încărcare a credențialelor le citește la pornire), iar log-ul portului serial confirmă numărul de rețele actualizate. În caz contrar, scrierea este omisă pentru a evita uzura prematură a memoriei flash. Rețelele noi devin active la următoarea repornire a dispozitivului, mecanismul evitând astfel întreruperea voluntară a unei sesiuni de monitorizare active.

5.1.10 Comunicația cu serverul și coada offline

Măsurătorile validate sunt acumulate și transmise periodic către backend, la un interval de ordinul zecilor de secunde, printr-o cerere HTTPS de tip POST către endpoint-ul de ingestie. Cererea este autentificată cu o cheie de dispozitiv inclusă într-un antet dedicat, iar conexiunea securizată TLS este validată pe baza unui pachet de certificate inclus în firmware. La un interval mai mare, de ordinul minutelor, dispozitivul trimite un mesaj de tip *heartbeat*, care raportează starea sa tehnică: puterea semnalului WiFi, memoria liberă, timpul de funcționare și numărul de măsurători aflate în coada de așteptare.

Atunci când transmiterea eșuează din cauza lipsei conexiunii, măsurătoarea este salvată într-o **coadă locală** păstrată în memoria nevolatilă (NVS) a microcontrollerului. Coada are o capacitate maximă fixă, iar la depășirea acesteia cele mai vechi înregistrări sunt suprascrise. La revenirea conexiunii, înregistrările aflate în așteptare sunt transmise treptat, într-un număr limitat la fiecare ciclu de comunicație, pentru a nu suprasolicita rețeaua și a nu întârzia achiziția curentă de date.

5.1.11 Interfața locală

Interacțiunea cu utilizatorul se realizează prin ecranul OLED, care afișează în timp real parametrii vitali și starea conexiunii, prin două LED-uri de stare și prin două butoane: un buton de pornire/oprire și un buton de panică, a cărui apăsare declanșează o cerere dedicată către backend, independent de valorile senzorilor.

5.1.12 Configurarea rețelei

Datele de conectare la rețeaua WiFi sunt păstrate în memoria nevolatilă a dispozitivului, fiind preîncărcate la *flash* cu o pereche SSID + parolă implicită folosită la prima pornire. Ulterior, aparținătorul poate înregistra prin aplicația web până la trei rețele WiFi suplimentare, transmise dispozitivului prin mecanismul de sincronizare descris în subsecțiunea anterioară. La fiecare pornire, dispozitivul încearcă pe rând rețelele salvate în NVS, în ordinea înregistrării lor; dacă niciuna nu răspunde, revine la credențialele implicite din *firmware*. Această abordare

respectă principiul simplității pentru utilizatorul vârstnic, prezentat în subcapitolul 2.1: după configurarea efectuată de aparținător din aplicația web, dispozitivul se reconectează automat la pornire în oricare dintre mediile pacientului (acasă, hotspot mobil, deplasare), fără a necesita vreo intervenție tehnică ulterioară.

5.2 Backend-ul și API-ul REST

Backend-ul a fost implementat în **ASP.NET Core 8**, organizat conform arhitecturii stratificate descrise în subcapitolul 4.1. Funcționalitatea este expusă printr-un ansamblu de controlere REST, fiecare grupând operațiile asociate unei resurse: autentificare, utilizatori, persoane monitorizate, măsurători, notificări, afecțiuni, invitații, precum și controlerele dedicate dispozitivelor IoT și interogării modelului de inteligență artificială.

5.2.1 Injecția de dependențe și accesul la date

Componentele backend-ului sunt înregistrate într-un container de **injecție de dependențe**, mecanism prin care serviciile și depozitele sunt furnizate automat claselor care le solicită, fără ca acestea să le instanțieze direct. Accesul la baza de date urmează șablonul **repository**: fiecare entitate are un depozit propriu, care încapsulează interogările Entity Framework Core și izolează restul aplicației de detaliile de persistență.

5.2.2 Autentificare și securitate

Autentificarea este gestionată prin token-uri JWT, validate la fiecare cerere pe baza unei chei secrete. Aplicația acceptă și autentificarea prin OAuth folosind un cont Google. O particularitate de implementare o reprezintă conexiunile SignalR, care, spre deosebire de cererile HTTP obișnuite, transmit token-ul prin șirul de interogare al adresei; backend-ul tratează explicit acest caz. Securitatea este întărită prin mai multe mecanisme complementare. **Limitarea ratei** (*rate limiting*) restricționează numărul de cereri de autentificare acceptate de la același client într-o fereastră de timp, atenuând atacurile de tip forță brută. Răspunsurile includ **antete HTTP de securitate**, precum X-Content-Type-Options, X-Frame-Options și Referrer-Policy, care reduc riscul atacurilor de tip *clickjacking* sau de interpretare greșită a conținutului. O **politică CORS** restrictivă permite accesul la API doar din partea aplicației web autorizate. Un **gestionar global de excepții** interceptează erorile neprevăzute și returnează un mesaj generic, fără a expune detalii interne care ar putea fi exploatare. De asemenea, întrucât backend-ul este găzduit în spatele unui *reverse proxy*, sunt procesate antetele transmise de acesta, pentru ca redirectionările și fluxul de autentificare OAuth să folosească corect protocolul HTTPS.

5.2.3 Serviciile de fundal

Pe lângă controlere, backend-ul include mai multe servicii de fundal care rulează independent de cereri. **Serviciul de monitorizare a alertelor** (AlertMonitorService) primește fiecare măsurătoare nou primită, ține un buffer cu valorile din ultimele minute pentru fiecare pacient și decide momentul exact de trimitere a notificărilor. În paralel, **serviciul de profilare a activității** (ActivityProfileService) construiește la prima nevoie un profil orar per pacient din ultimele 14 zile de măsurători și verifică, la fiecare nouă măsurătoare, dacă eticheta de activitate primită de la dispozitiv abate de la tiparul obișnuit; abaterile semnificative declanșează notificările comportamentale descrise în subcapitolul 4.4. **Serviciul de raportare zilnică** (DailyReportBackgroundService) rulează o dată pe zi, la miezul nopții în fusul orar local configurat, și trimite prin email un raport sintetic al zilei precedente utilizatorilor care au activat această opțiune. **Serviciul de retenție a datelor** (RetentionCleanupBackgroundService) rulează tot zilnic și execută ștergerea fizică a măsurătorilor, notificărilor și agregărilor mai vechi

decât perioada de retenție configurată pentru fiecare pacient. La pornirea aplicației se execută, de asemenea, popularea inițială a bazei de date cu rolurile și datele de bază.

5.2.4 Contractul cu clientul și optimizări

Comunicarea cu aplicația web folosește obiecte de transfer de date (DTO) definite în proiectul partajat *Shared*, ceea ce garantează că backend-ul și clientul folosesc exact aceeași structură a datelor și elimină erorile de nepotrivire a câmpurilor. Pentru a reduce volumul transferat prin rețea și a accelera încărcarea aplicației, răspunsurile sunt comprimate folosind algoritmi Brotli și Gzip, iar interogările frecvente către baza de date sunt accelerate prin indexurile descrise în subcapitolul 4.3.

5.2.5 Validarea cererilor

Fiecare cerere primită de API este validată înainte de a fi procesată: sunt verificate prezența câmpurilor obligatorii, încadrarea valorilor în intervale plauzibile și dreptul utilizatorului de a accesa resursa solicitată. O cerere invalidă este respinsă cu un mesaj explicativ, fără a ajunge la stratul de persistență. Pentru endpoint-urile dispozitivelor IoT, validarea include suplimentar verificarea cheii de dispozitiv transmisă în antetul cererii și existența unei persoane monitorizate asociate numărului de serie.

5.3 Microserviciul de inteligență artificială

Microserviciul de inteligență artificială a fost implementat în limbajul **Python**, folosind framework-ul web **FastAPI**. La pornire, serviciul încarcă din fișierul de artefacte modelul Random Forest antrenat, împreună cu obiectele auxiliare necesare (scalatorul Min-Max și codificatorul de etichete), serializate anterior cu biblioteca joblib.

5.3.1 Antrenarea modelului

Modelul de inteligență artificială a fost antrenat separat de aplicație, printr-un proces reproductibil. Sunt încărcate mai întâi cele trei seturi de date publice menționate în subcapitolul 2.3, pe baza cărora este generat un set de date sintetic, prin eșantionare cu adăugare de zgomot gaussian; setul cuprinde câteva mii de exemple normale și câteva mii de exemple corespunzătoare stărilor de alertă și critice. Datele sunt împărțite în subseturi de antrenare și de testare, componentele de mișcare sunt normalizate, iar modelul Random Forest este antrenat și evaluat. La final, modelul împreună cu obiectele auxiliare (scalatorul și codificatorul de etichete) sunt serializate într-un singur fișier de artefacte, exact fișierul pe care microserviciul îl încarcă la pornire. Astfel, etapa de antrenare, costisitoare și executată o singură dată, este complet separată de etapa de inferență, rapidă și executată la fiecare măsurătoare primită.

5.3.2 Endpoint-urile serviciului

Serviciul expune trei endpoint-uri: unul de verificare a stării (folosit de mecanismul de *health-check* al containerului), unul care returnează informații despre model și endpoint-ul principal de predicție. Backend-ul în C# apelează acest endpoint printr-un client HTTP configurat cu un timp de așteptare scurt, astfel încât o eventuală indisponibilitate a serviciului AI să nu afecteze fluxul principal.

5.3.3 Logica hibridă de decizie

În interiorul endpoint-ului de predicție, datele primite sunt mai întâi validate: o măsurătoare cu valori în afara intervalelor fiziologic plauzibile este respinsă, evitând astfel o predicție lipsită de sens. Componentele de mișcare sunt apoi normalizate cu scalatorul Min-Max, iar modelul Random Forest produce o predicție și un set de probabilități asociate celor trei clase.

Peste predicția modelului se aplică un strat de **reguli medicale**, cu prioritate față de model. Regulile absolute decid singure în cazurile clare; de exemplu, o saturație a oxigenului sub pragul critic determină starea CRITICAL indiferent de ce a prezis modelul. Un set de reguli cumulative analizează simultan toți parametrii, fiecare abatere contribuind la un scor de severitate pe baza căruia, împreună cu starea modelului, se stabilește starea finală. Modificatorii specifici afecțiunilor preexistente, prezentați în subcapitolul 4.4, ajustează la rândul lor evaluarea. Această stratificare asigură că situațiile clare din punct de vedere clinic sunt tratate corect, iar explicația textuală returnată indică exact ce parametru sau ce regulă a determinat decizia.

5.4 Frontend-ul web

Aplicația web a fost implementată în **Blazor WebAssembly**, sub forma unei aplicații de tip SPA. Interfața este organizată în pagini corespunzătoare funcționalităților principale: autentificare și înregistrare, tabloul de bord, lista persoanelor monitorizate, vizualizarea detaliată a unei persoane, istoricul măsurărilor, notificările, setările și paginile de administrare. Punctul de intrare al aplicației îl reprezintă paginile de *autentificare* (figura 13) și de *înregistrare* (figura 14); utilizatorul poate alege între autentificarea clasică pe bază de email și parolă și autentificarea delegată prin contul Google (OAuth 2.0).

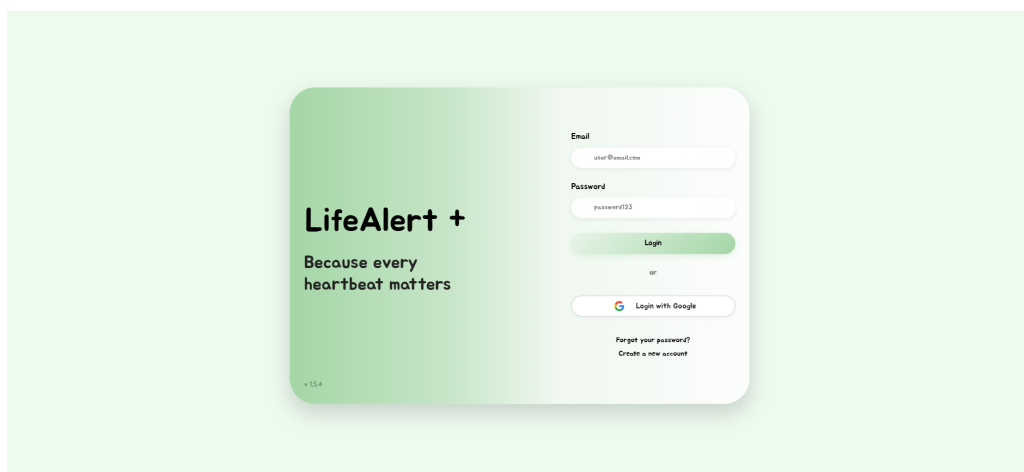


Figura 13: Pagina de autentificare a aplicației LifeAlert+: câmpuri pentru email și parolă, plus butonul de autentificare prin Google.

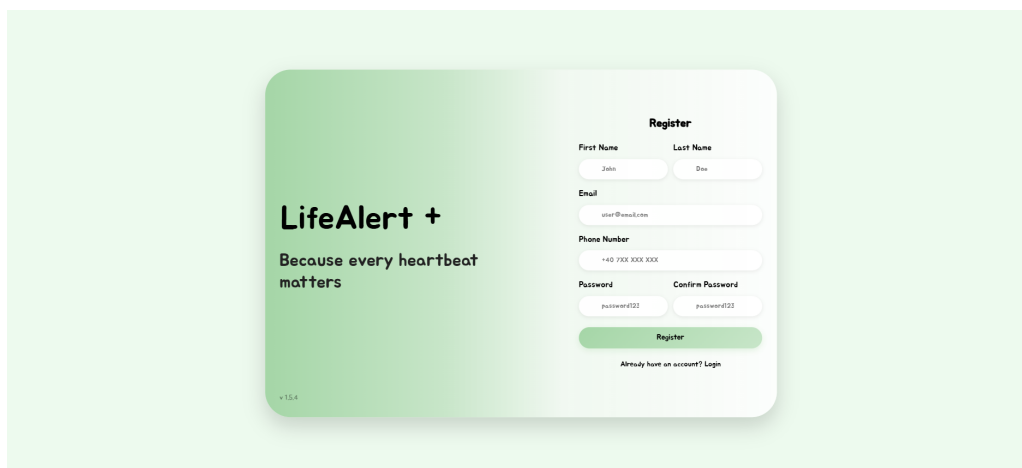


Figura 14: Pagina de înregistrare: utilizatorul completează datele personale, email-ul, numărul de telefon și parola.

5.4.1 Comunicarea cu backend-ul

Comunicarea cu API-ul se realizează prin clienți dedicați, construiți peste componenta `HttpClient`. Token-ul de autentificare este atașat automat fiecărei cereri printr-un gestionar de mesaje (*message handler*) interpus, care elimină necesitatea adăugării manuale a antetului de autorizare. Pentru actualizarea în timp real, aplicația menține o conexiune `SignalR` către backend, prin care primește notificările push fără a fi necesară reîncărcarea paginii.

5.4.2 Autorizarea pe roluri și accesibilitatea

Accesul la pagini este controlat în funcție de rolul utilizatorului: paginile de administrare sunt disponibile doar administratorilor, iar fiecare aparținător sau medic vede exclusiv persoanele monitorizate la care are drept de acces. Tentativele de acces neautorizat sunt redirectionate către pagina de autentificare. Întrucât aplicația se adresează inclusiv unui public mai puțin familiarizat cu tehnologia, interfața include opțiuni de accesibilitate, precum selectarea limbii (română sau engleză), comutarea limbii fiind reflectată în timp real pe toate paginile, precum și un **asistent conversațional integrat** (figura 15), care răspunde la întrebări frecvente despre utilizarea aplicației, pragurile semnelor vitale și afecțiunile medicale.

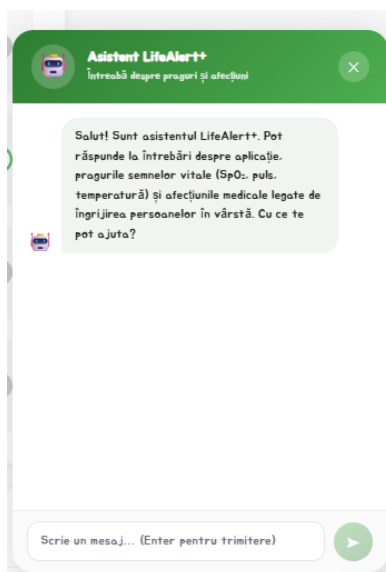


Figura 15: Asistentul conversațional integrat răspunde la întrebări despre utilizarea aplicației și despre praguri / afecțiuni medicale.

5.4.3 Organizarea interfeței și gestionarea stării

Interfața este construită din componente Blazor reutilizabile, anume antetul, widget-urile și notificările de tip *toast*, combinate în pagini. Starea partajată între pagini, precum datele utilizatorului autentificat sau persoana monitorizată selectată, este menținută de servicii dedicate, înregistrate în containerul de injecție de dependențe al aplicației client. Astfel, informațiile încărcate o singură dată sunt disponibile în întreaga aplicație, fără a fi nevoie de cereri repetate către server la fiecare schimbare de pagină. Pagina principală a aparținătorului este lista persoanelor monitorizate (figura 16), unde fiecare pacient apare sub forma unui card cu starea curentă a parametrilor vitali, ultima actualizare, indicatorul GPS și statusul detectorului de căderi.

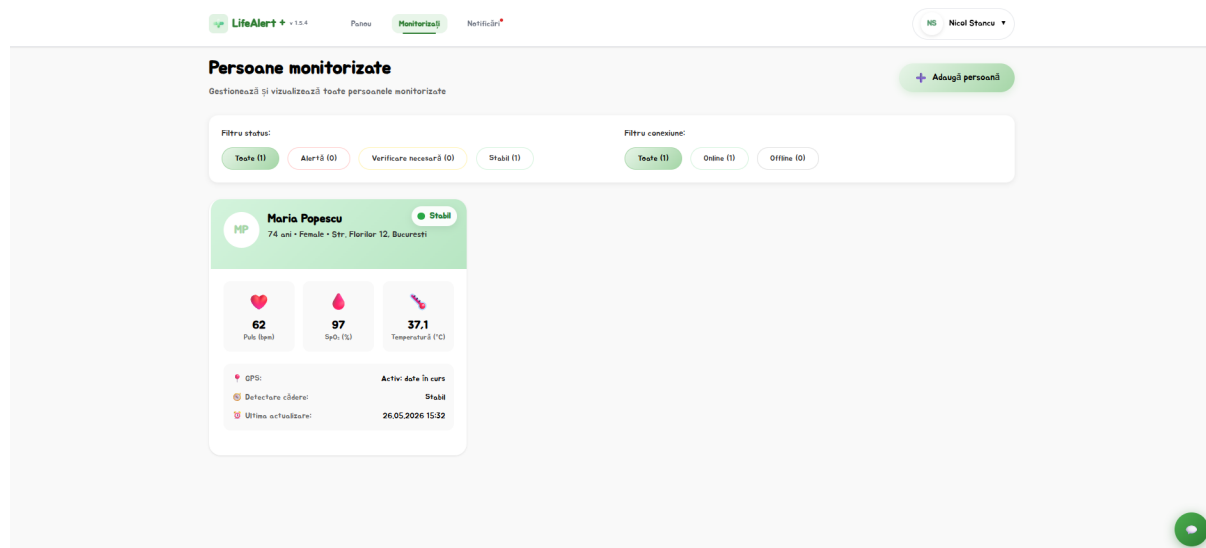


Figura 16: Pagina cu persoanele monitorizate: cardul rezumă vitalele curente (puls, SpO_2 , temperatură), starea (Stabil / Alertă / Verificare necesară), poziția GPS și momentul ultimei măsurători.

Tot din această pagină se accesează **modalul de editare a profilului pacientului** (figura 17), prin care aparținătorul configurează pragurile personalizate ale fiecărui parametru vital, frecvența de actualizare a dispozitivului și perioada de retenție a datelor. Suplimentar, modalul de **configurare a rețelelor WiFi** (figura 18) permite înregistrarea a până la trei rețele pe care dispozitivul portabil le va încerca la conectare.

Editare profil

Adresă
Str. Florilor 12, Bucuresti

Număr serial dispozitiv
ESP-SEED-001

Intervale vitale personalizate (lasă gol pentru valorile implicite ale contului)

Ritm cardiac minim (bpm) 60	Ritm cardiac maxim (bpm) 100
Temperatură minimă (°C) 36	Temperatură maximă (°C) 37,5
SpO ₂ minim (%) (%) 95	SpO ₂ maxim (%) (%) 100

Frecvența actualizării (lasă gol pentru valoarea implicită: 30 s)
▼

Păstrare date
Cât timp se păstrează măsurătorile, notificările și istoricul acestui pacient. Datele mai vechi sunt șterse definitiv zilnic.
Implicit (1 an) ▼

Anulare Salvare

Figura 17: Modalul de editare a profilului unei persoane monitorizate, cu pragurile personalizate pentru ritm cardiac, temperatură și SpO₂, frecvența de actualizare și perioada de retenție.

Rețele WiFi pentru dispozitiv

Adaugă până la 3 rețele pe care dispozitivul le va încerca la conectare. Modificările se aplică la următoarea repornire a dispozitivului.

Nicio rețea configurată. Dispozitivul va folosi rețeaua implicită.

Adaugă o rețea

Nume rețea (SSID)
ex. WifiAcasa

Parolă
Lasă gol pentru rețele deschise

+ Adaugă

Închide

Figura 18: Modalul de configurare a rețelelor WiFi pentru dispozitivul portabil al pacientului.

5.4.4 Vizualizarea datelor și funcționalitatea PWA

Reprezentarea grafică a istoricului parametrilor vitali este realizată direct în SVG, prin curbe Bézier monotone (Fritsch-Carlson) trasate peste o grilă cu axă temporală. Cele două grafice (puls și temperatură) sunt așezate într-un container cu **scroll orizontal sincronizat** și un control de **zoom** (1x–4x): wrapper-ul rămâne la lățimea cardului, iar SVG-ul intern se lungește, astfel încât citirile dense să nu se mai suprapună (figura 19). La trecerea mouse-ului peste oricare zonă a graficului, un tooltip arată valoarea exactă și ora la cel mai apropiat punct înregistrat.



Figura 19: Graficele zilnice de puls și temperatură, cu tooltip-ul activ la hover (afișează valoarea, de exemplu 82 bpm la 19:16, și ora) și scrollbar orizontal sincronizat sub fiecare grafic.

Localizarea pacientului în scenarii de alertă este realizată tot prin interoperabilitate JavaScript, integrând harta Google Maps direct în vizualizarea persoanei monitorizate (figura 20). Marcatorul este actualizat automat la fiecare nouă măsurătoare cu fix GPS valid. Pentru a evita expunerea cheii API în codul client, aceasta nu este inclusă în pachetul aplicației, ci este obținută la nevoie de la backend printr-un endpoint dedicat (servit doar către cereri autentificate), apoi predată bibliotecii de hărți. Exportul rapoartelor în format PDF folosește același mecanism de interoperabilitate.

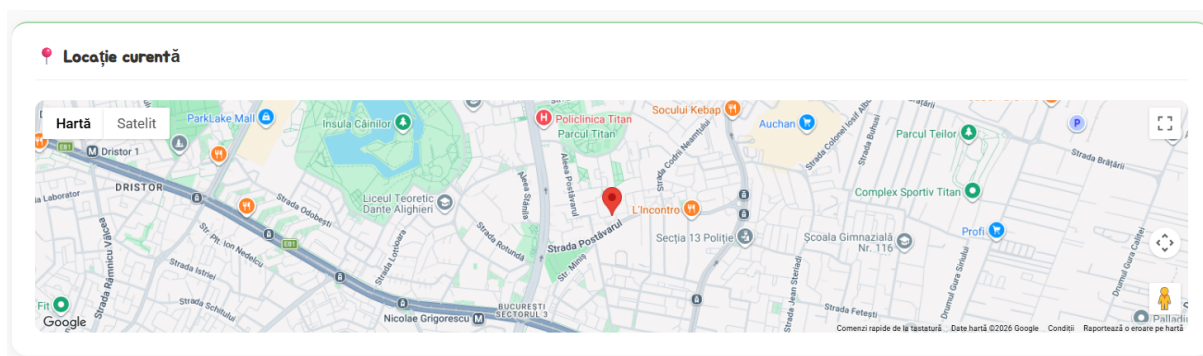


Figura 20: Vizualizarea pe hartă a poziției curente a pacientului în pagina sa de detalii.

Aplicația este configurată ca **PWA**: include un service worker, un fișier manifest care permite instalarea pe dispozitiv și o pagină dedicată modului offline.

5.4.5 Tabloul de bord și paginile aparținătorului

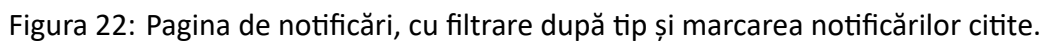
Pagina de start a aparținătorului după autentificare este **tabloul de bord** (figura 21), care oferă o privire de ansamblu: numărul total de persoane monitorizate, câte se află în stare de alertă, câte sunt stabile, câte sunt offline și numărul de măsurători din ziua curentă, urmate de carduri rezumat pentru cele mai recent actualizate persoane. Tabloul de bord se reîmprospătează periodic, astfel încât starea afișată să rămână actuală fără reîncărcarea manuală a paginii.

Pagina de **notificări** (figura 22) listează cronologic alertele și mesajele primite, cu posibilitatea de filtrare după tip și de marcare ca citite; un indicator în antet semnalează notificările necitite. Pagina de **istoric al măsurătorilor** (figura 23) prezintă tabelar măsurătorile brute ale unei persoane, paginat, util pentru o consultare detaliată dincolo de reprezentarea grafică.

Pagina de **profil** (figura 24) permite utilizatorului să își editeze datele personale, fotografia de profil și parola, iar pagina de **setări** (figura 25) grupează preferințele de cont: limba interfeței, preferințele de notificare (descrise în subcapitolul 5.5), pragurile vitale implicite și prima zi a săptămânii folosită în graficele săptămânale.

Captură: tabloul de bord al aparținătorului (figuri/dashboard.png) — de inserat.

Figura 21: Tabloul de bord al aparținătorului, cu statistici de ansamblu și carduri pentru persoanele recent actualizate.



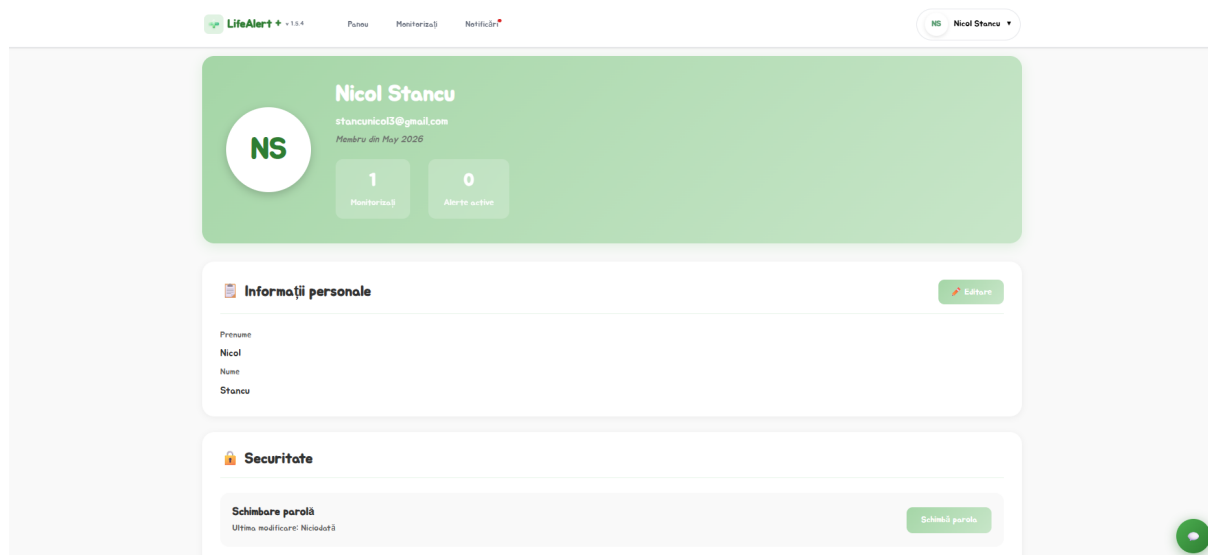


Figura 24: Pagina de profil a utilizatorului.

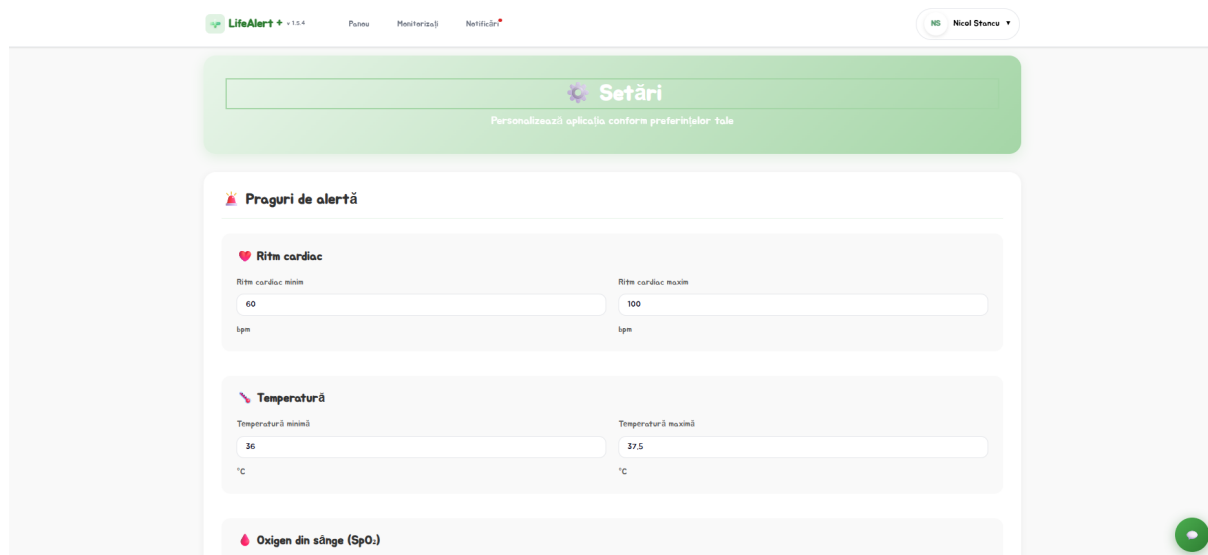


Figura 25: Pagina de setări a contului, cu preferințele de limbă, notificare și pragurile vitale implicite.

5.4.6 Paginile informative și suportul

Aplicația include și două pagini cu rol informativ, accesibile din meniul principal. Pagina **Despre noi** (figura 26) prezintă, sub formă de carduri, misiunea proiectului, problema pe care o adresează și principalele caracteristici. Pagina de **Support** (figura 27) oferă o imagine a dispozitivului, datele de contact prin email, o listă de verificări de bază (dispozitiv încărcat, repornit, conexiune la internet) și o secțiune de întrebări frecvente cu soluții pentru problemele comune (de exemplu, lipsa datelor de la dispozitiv).

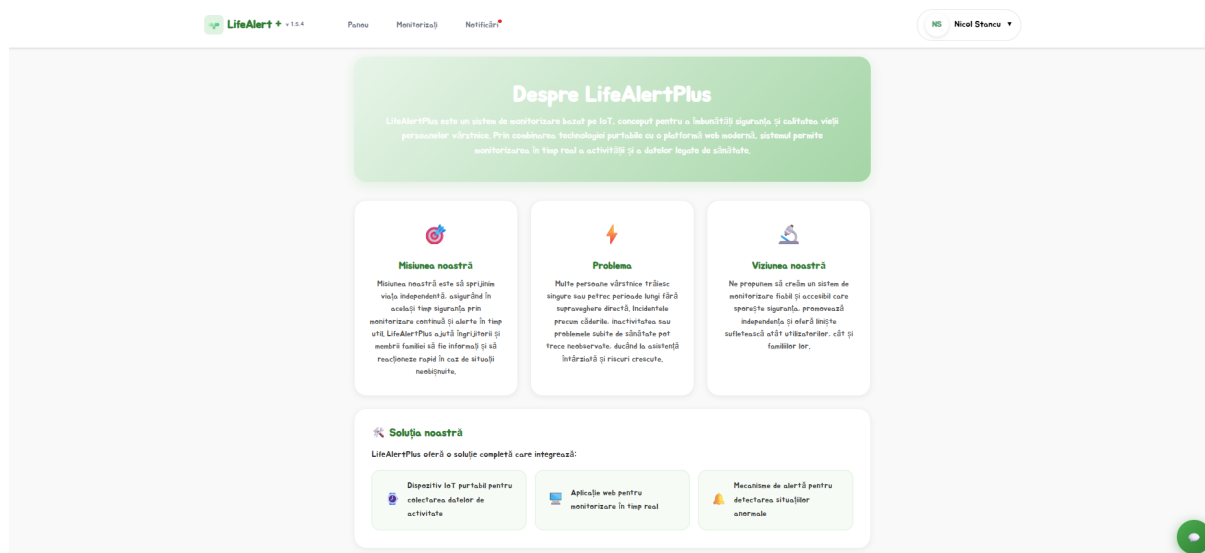


Figura 26: Pagina „Despre noi”, cu misiunea, problema adresată și caracteristicile proiectului.

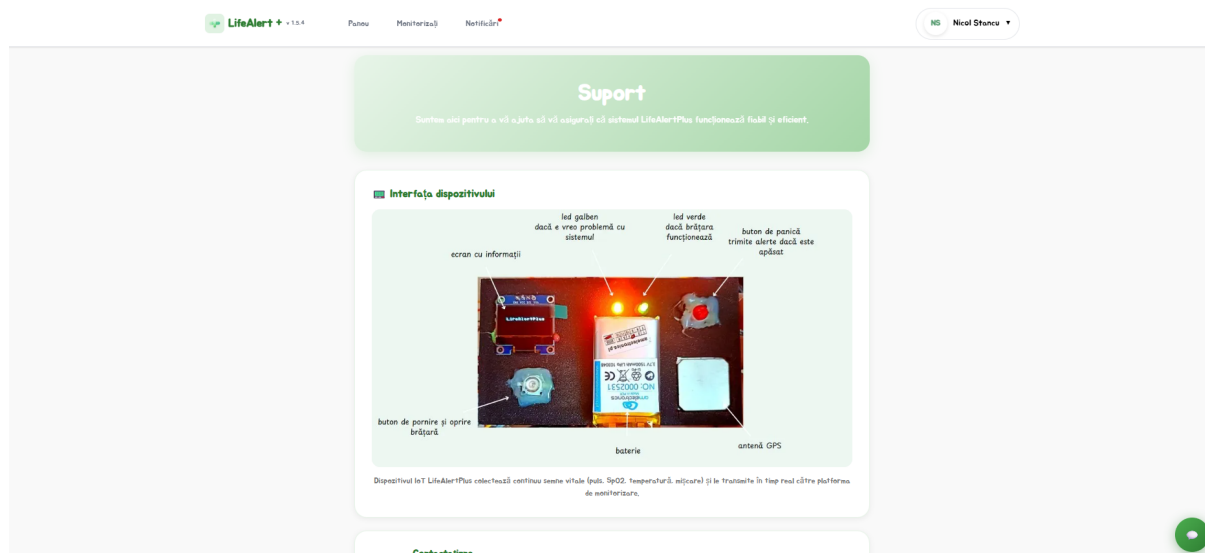


Figura 27: Pagina de suport, cu datele de contact, lista de verificări și întrebările frecvente.

5.4.7 Interfața de administrare

Pe lângă interfața destinată aparținătorilor și medicilor, aplicația include un set de pagini accesibile exclusiv rolului de **administrator**, protejate prin verificarea rolului din token-ul JWT la fiecare navigare. Acestea cuprind:

- **Tabloul de bord de administrare** (figura 28), care prezintă statistici agregate la nivelul întregului sistem, calculate în timp real din datele reale: numărul total de utilizatori, numărul de persoane monitorizate, câte dispozitive sunt online sau offline, numărul de alerte active și numărul de măsurători din ziua curentă, alături de o listă a notificărilor recente din sistem.
- **Gestionarea utilizatorilor**, prin care administratorul poate vizualiza conturile, le poate edita sau șterge și poate consulta persoanele monitorizate asociate fiecăruia.
- **Jurnalul de activitate** (figura 29) și **jurnalul de erori** (figura 30), două vizualizări tabelare cu

filtrare și căutare, destinate urmării evenimentelor de cont și de sistem, respectiv a erorilor și avertismentelor backend-ului. În stadiul actual, aceste două vizualizări prezintă date reprezentative, structura lor fiind pregătită pentru conectarea ulterioară la un mecanism de audit persistent.

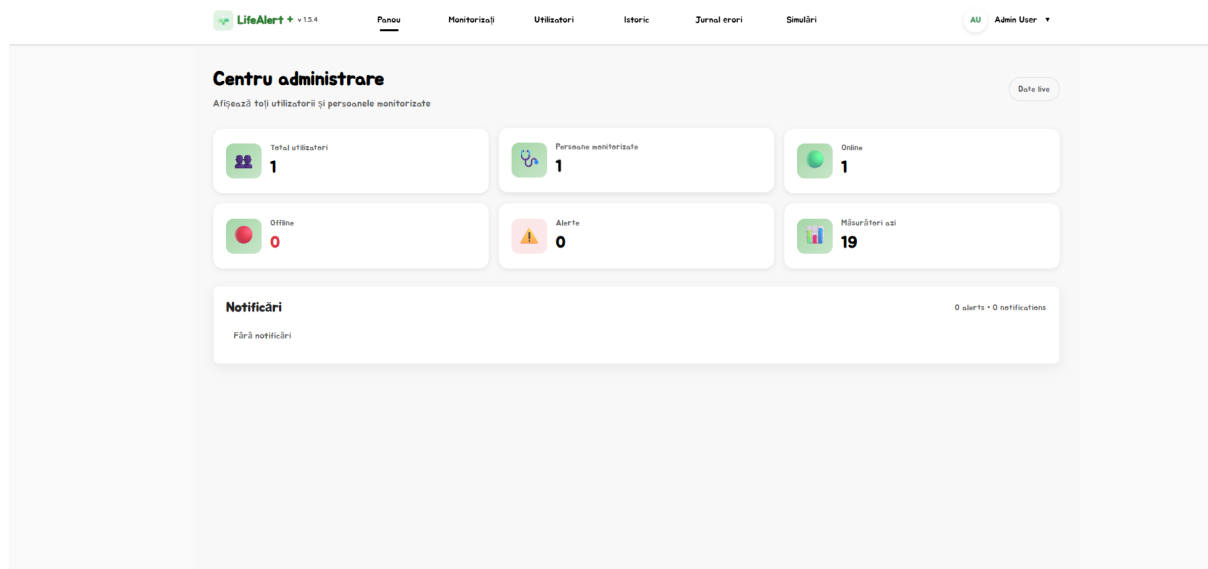


Figura 28: Tabloul de bord de administrare, cu statisticile agregate la nivelul întregului sistem.

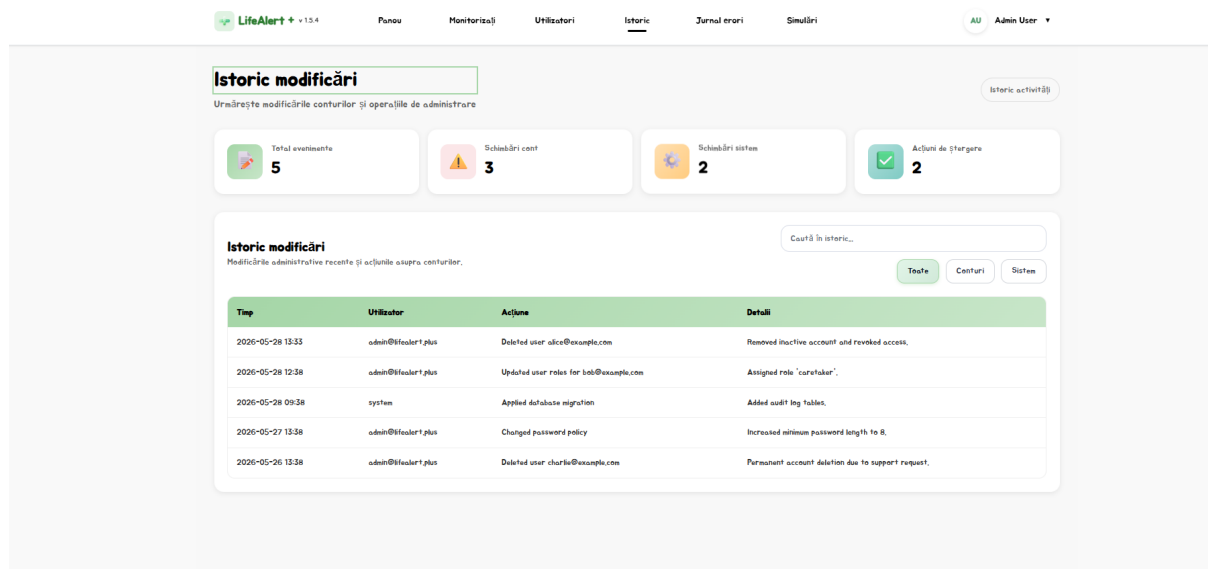


Figura 29: Jurnalul de activitate (evenimente de cont și de sistem), cu filtrare și căutare.

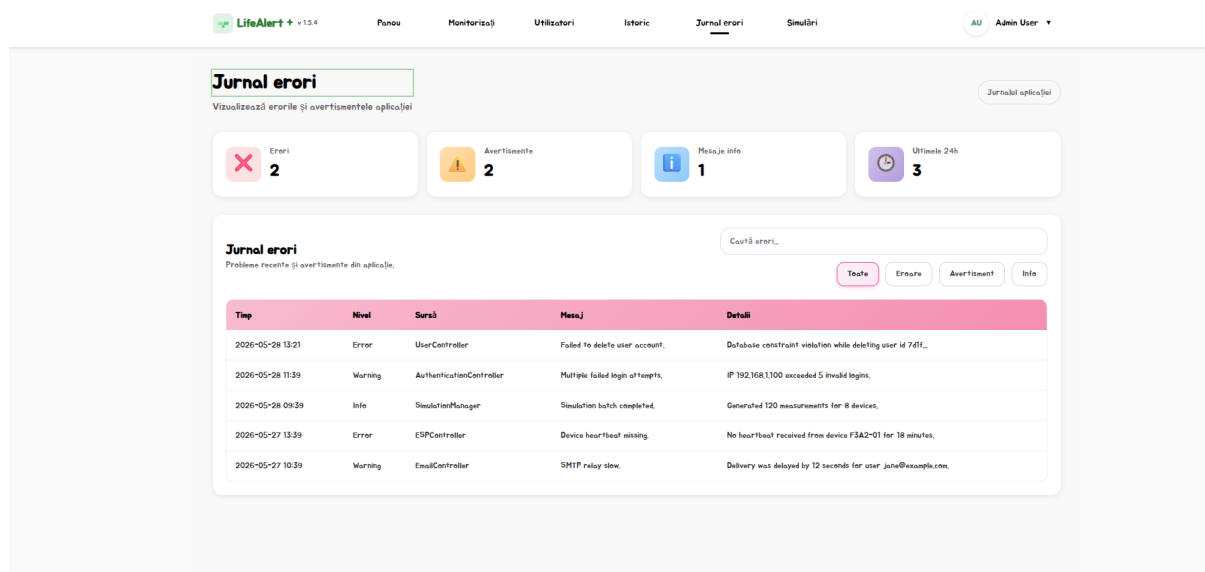


Figura 30: Jurnalul de erori al backend-ului, cu filtrare după nivel (eroare, avertisment, informativ).

5.4.8 Vizualizarea medicului

Medicul invitat de un aparținător (prin mecanismul descris în subcapitolul 4.4) primește acces **în regim de citire** la datele pacientului respectiv, printr-o pagină dedicată similară celei a aparținătorului, dar fără controalele de editare a profilului, de configurare a rețelelor WiFi sau de modificare a pragurilor (figura 31). Medicul poate consulta starea curentă, istoricul graficelor de parametri vitali, profilul de activitate și poate primi prin email un export structurat al măsurătorilor pentru un interval de timp selectabil, facilitând astfel evaluarea clinică în afara aplicației.

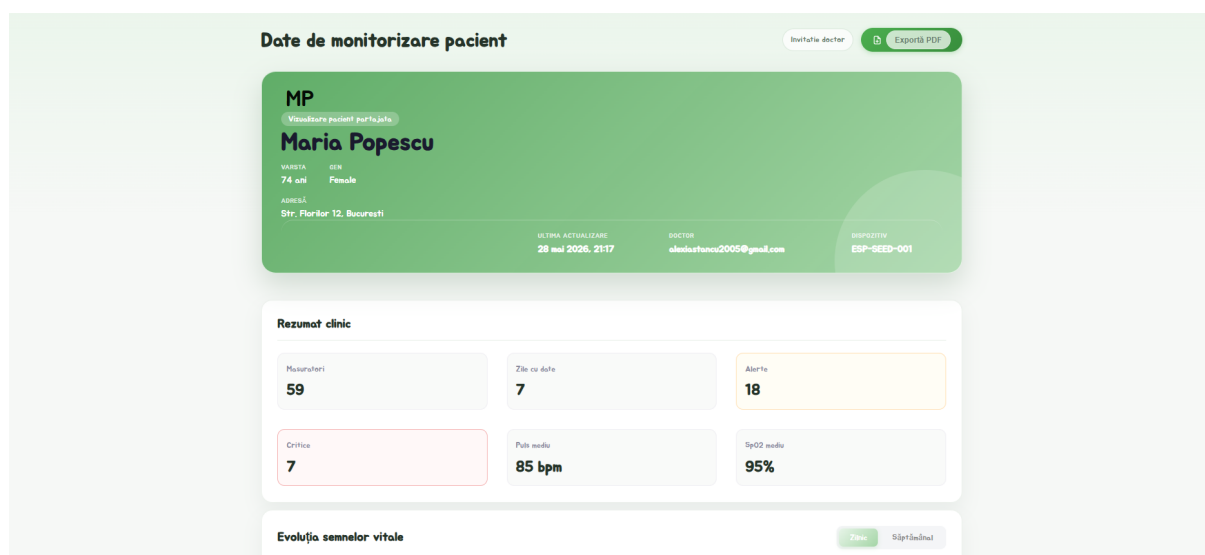


Figura 31: Vizualizarea în regim de citire a datelor pacientului, disponibilă medicului invitat.

5.5 Sistemul de notificări

Sistemul de notificări implementează alertarea multi-canal descrisă conceptual în subcapitolul 2.4. Logica de declanșare este coordonată de serviciul de monitorizare a alertelor, care decide când și prin ce canale trebuie transmisă o notificare, ținând cont de preferințele fiecărui utilizator. Aceste preferințe sunt expuse într-o pagină dedicată de setări (figura 32), unde fiecare canal (email, push, SMS) și raportul zilnic pot fi activate sau dezactivate independent printr-un comutator.

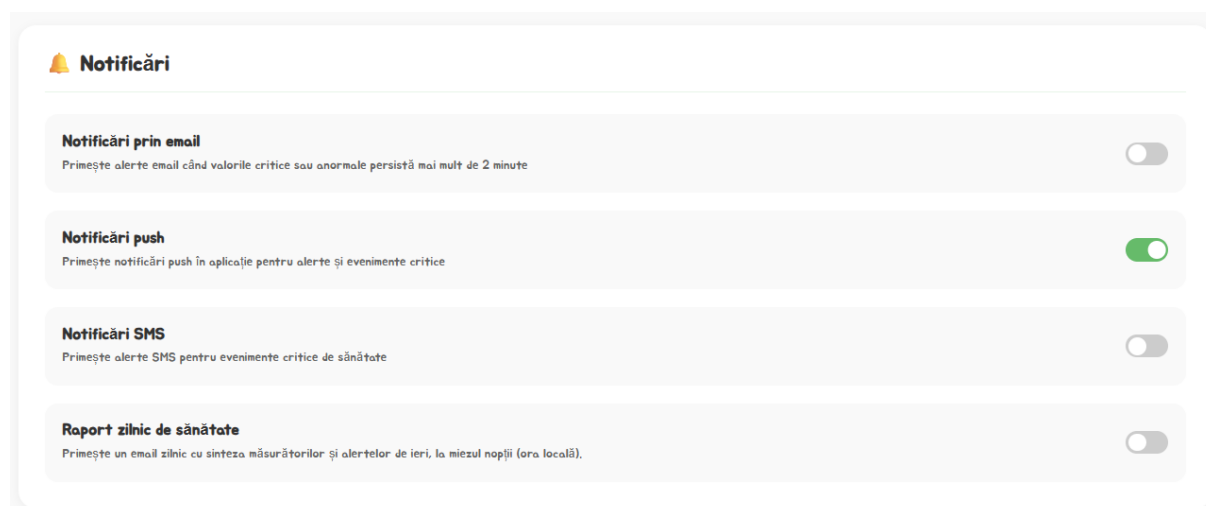


Figura 32: Pagina de setări pentru notificări: comutatoarele permit activarea independentă a email, push, SMS și raport zilnic.

5.5.1 Temporizarea alertelor

Pentru a evita alertele false, serviciul de monitorizare aplică mai multe temporizări. O stare anormală trebuie să persiste o durată minimă, anume pragul de persistență, de aproximativ două minute pentru stările critice, înainte de a genera o notificare, eliminând valorile tranzitorii. La declanșare, alerta este transmisă simultan prin toate canalele activate de utilizator (push, email și SMS), fiecare alertă fiind însoțită de SMS dacă acest canal este selectat; SMS-ul nu este întârziat suplimentar față de celelalte canale. Atâta vreme cât starea critică persistă, notificarea este retrimisă la fiecare două minute; pentru alertele non-critice, un interval de inhibare (*cooldown*) împiedică retransmiterea aceleiași alerte pentru aceeași persoană, eliminând riscul inundării aparținătorului cu mesaje. Pentru aplicarea acestor reguli, serviciul menține în memorie, pentru fiecare persoană monitorizată, un buffer cu măsurătorile din ultimele minute.

5.5.2 Notificările push

Notificările push sunt livrate prin **SignalR**, printr-un hub la care clienții se conectează după autentificare. Fiecare utilizator este asociat unui grup propriu, identificat prin identificatorul său, astfel încât o notificare să poată fi direcționată exact către destinatarul vizat. Mesajele sunt afișate instantaneu în interfață sub forma unui *toast* colorat în funcție de severitate (figura 33) și sunt salvate, totodată, în baza de date, pentru a putea fi consultate ulterior din pagina de notificări.

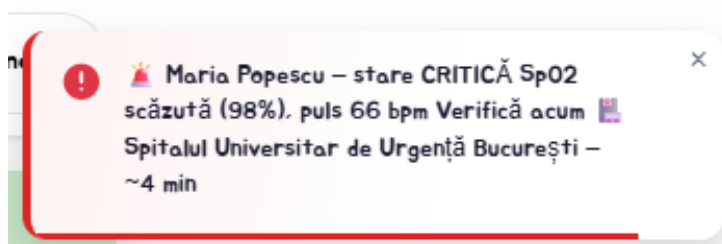


Figura 33: Notificare push afișată în interfața web prin SignalR, imediat ce alerta este declanșată pe server.

5.5.3 SMS și email

Trimiterea SMS-urilor este realizată prin integrarea cu platforma de comunicație în cloud **Twilio**, apelată prin biblioteca oficială a acesteia. Mesajul SMS (figura 34) conține tipul alertei, numele persoanei monitorizate și valorile vitale relevante, format compact pentru a se încadra într-un singur segment. Email-urile (figura 35), atât notificările, cât și mesajele tranzacționale precum confirmarea contului sau resetarea parolei, sunt trimise printr-un server SMTP, folosind un *template* HTML cu identitatea vizuală a aplicației. Fiecare utilizator poate activa sau dezactiva independent fiecare canal din pagina de setări, iar sistemul respectă aceste preferințe la livrarea fiecărei alerte.



Figura 34: Alertă livrată prin SMS pe telefonul aparținătorului, prin platforma Twilio.



Figura 35: Email de alertă cu *template* HTML dedicat, livrat printr-un server SMTP configurat în aplicație.

5.5.4 Raportul zilnic

Pe lângă alertele declanșate de situații de risc, sistemul generează, pentru utilizatorii care au activat această opțiune dintr-un comutator dedicat din pagina de setări, un **raport zilnic** de sănătate. Serviciul de fundal corespunzător rulează o dată pe zi, la miezul nopții în fusul orar local configurat în setările serverului (de exemplu, Europe/Bucharest), agregă măsurătorile zilei precedente, anume valorile minime, maxime și medii ale parametrilor vitali, numărul de căderi și de alerte, și trimite sinteza prin email folosind un **template HTML dedicat**, distinct de cel al notificărilor de alertă. Acest mecanism oferă aparținătorului o imagine de ansamblu chiar și în zilele fără evenimente, fără a fi necesară consultarea activă a aplicației.

5.5.5 Feedback pentru alarme false

Pentru a îmbunătăți calitatea sistemului de alertare în timp și pentru a oferi date concrete despre rata reală de alerte false, aplicația implementează o buclă de feedback: atunci când o stare de alertă se rezolvă (vitalele revin la normal), ultima notificare a episodului este marcată automat în baza de date ca *așteptând răspuns*. La următoarea vizită a aparținătorului în aplicație, o componentă plasată în layout-ul principal verifică existența unor notificări marcate și afișează un popup care întreabă explicit dacă alerta a fost reală sau falsă. Răspunsul (*Da, reală*, *Nu, alarmă falsă* sau *Întreabă-mă mai târziu*) este înregistrat pe notificare, iar prima opțiune fixează răspunsul definitiv, în timp ce ultima lasă popup-ul să reapară la sesiunea următoare.

Datele etichetate astfel permit, ulterior, calcularea ratei reale de alerte false raportate de utilizatori, metrică folosită și în capitolul de testare al lucrării.

6 Testare și rezultate

Acest capitol prezintă metodologia de testare aplicată sistemului și rezultatele obținute. Sunt urmărite patru aspecte: corectitudinea funcțională, performanța modelului de inteligență artificială, latența sistemului și fiabilitatea mecanismului de alertare.

[Notă: valorile numerice marcate mai jos cu paranteze drepte trebuie completate cu rezultatele măsurate efectiv în timpul testării. Metodologia este descrisă complet; doar cifrele finale rămân de inserat.]

6.1 Metodologia de testare

Testarea sistemului a fost realizată pe mai multe planuri. **Testarea funcțională** a vizat verificarea fiecărui flux descris în subcapitolul 4.4, anume înregistrarea utilizatorilor, asocierea unei persoane monitorizate, recepția măsurătorilor, declanșarea alertelor și livrarea notificărilor.

Pentru testarea fluxurilor care depind de dispozitivul fizic a fost utilizat un **mod de simulare**, integrat în aplicație, care permite generarea de măsurători cu valori controlate, fără a fi necesar hardware-ul real. Acest mecanism a permis reproducerea sistematică a scenariilor de risc, anume valori critice de SpO_2 , tahicardie, febră și imobilitate, urmate de verificarea reacției corecte a sistemului în fiecare caz.

Performanța modelului de inteligență artificială a fost evaluată separat, pe setul de date, folosind tehnicile de validare descrise mai jos.

6.2 Acuratețea datelor și performanța modelului AI

Performanța modelului Random Forest a fost estimată prin **validare încrucișată stratificată în 5 falduri**, metodă care împarte repetat setul de date în subseturi de antrenare și testare, păstrând în fiecare fald distribuția claselor NORMAL, ALERT și CRITICAL. Au fost urmărite acuratețea, scorul F1 (în variantele macro și ponderată) și aria de sub curba ROC (AUC-ROC), calculată în regim *one-vs-rest*.

Setul de date utilizat la antrenare și evaluare a fost construit sintetic, pornind de la sursele publice menționate în subcapitolul 2.3. El cuprinde câteva mii de exemple corespunzătoare stării normale și câteva mii de exemple corespunzătoare stărilor ALERT și CRITICAL, generate în mod echilibrat astfel încât modelul să dispună de suficiente cazuri din fiecare clasă. Caracterul stratificat al validării garantează că această proporție este păstrată în fiecare fald, evitând estimările optimiste care ar putea apărea dacă o clasă rară ar fi insuficient reprezentată în subsetul de testare.

Rezultatele obținute sunt sintetizate în tabelul 4.

Metrică	Medie	Abatere standard
Acuratețe	[]	[]
Scor F1 (macro)	[]	[]
Scor F1 (ponderat)	[]	[]
AUC-ROC (one-vs-rest)	[]	N/A

Tabela 4: Rezultatele validării încrucișate stratificate în 5 falduri.

Pentru a verifica absența supraînvățării (overfitting), a fost comparată acuratețea pe datele de antrenare cu cea pe datele de testare; o diferență mică între cele două indică o bună capacitate de generalizare a modelului (*diferența măsurată: []*).

Comportamentul modelului pe fiecare clasă a fost analizat prin **matricea de confuzie** (figura 36), care evidențiază eventualele confuzii între stările apropiate. Importanța parametrilor de intrare a fost evaluată prin metoda **SHAP** (figura 37), care a confirmat că saturația oxigenului, frecvența cardiacă și temperatura corporală sunt factorii dominanți în decizia modelului, în concordanță cu criteriile clinice de triaj.

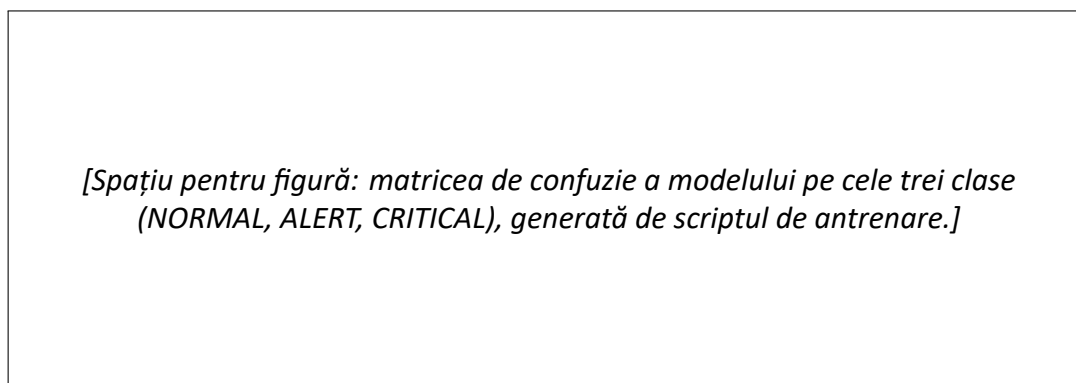


Figura 36: Matricea de confuzie a modelului Random Forest.

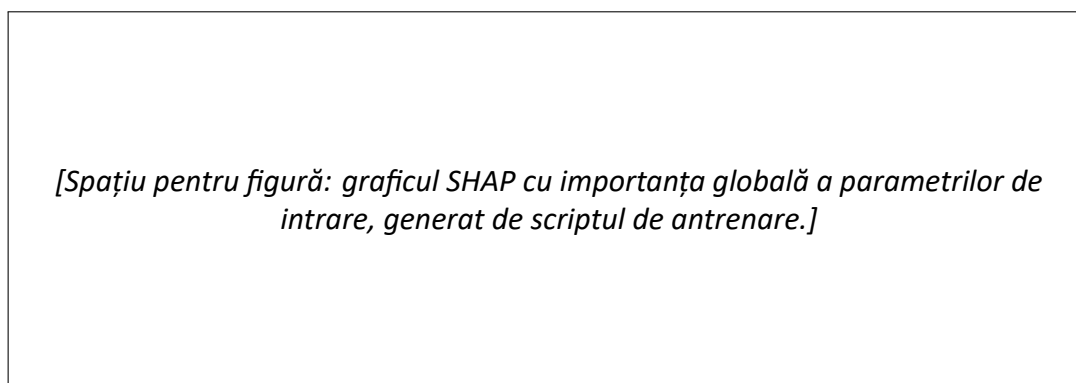


Figura 37: Importanța parametrilor de intrare (analiză SHAP).

6.3 Latența sistemului

Latența reprezintă intervalul de timp scurs între achiziția unei măsurători pe dispozitiv și afișarea sau alertarea acesteia (obiectivul O5). Au fost măsurate două intervale: timpul *dispozitiv* → *afișare*, de la citirea senzorilor până la actualizarea tabloului de bord, și timpul *dispozitiv* → *alertă*, de la achiziție până la livrarea notificării push.

Latența totală este suma mai multor componente: intervalul de transmitere configurat pe dispozitiv, timpul de procesare pe backend, timpul de răspuns al microserviciului AI și timpul de livrare prin SignalR. Valorile măsurate au fost: timp *dispozitiv* → *afișare* de aproximativ [] secunde și timp *dispozitiv* → *alertă* de aproximativ [] secunde, confirmând funcționarea în regim apropiat de timp real.

6.4 Fiabilitatea alertelor

Fiabilitatea mecanismului de alertare a fost evaluată din două perspective: capacitatea de a semnaliza situațiile reale de risc și capacitatea de a evita alertele false. Folosind modul de simulare, au fost generate scenarii de risc cunoscute și s-a verificat dacă sistemul declanșează alerta corespunzătoare. În paralel, au fost generate valori normale cu variații tranzitorii, pentru a verifica dacă pragul de persistență elimină alertele false.

Rezultatele au arătat că, din [] scenarii de risc testate, sistemul a declanșat corect alerta în [] cazuri, iar pragul de persistență a redus numărul de alerte false cu []% față de o variantă fără acest mecanism. A fost testat separat și comportamentul la pierderea conexiunii: măsurătorile acumulate în coada locală a dispozitivului au fost transmise integral și în ordine corectă la revenirea semnalului, fără pierderi de date.

7 Concluzii și perspective

7.1 Concluzii

Lucrarea de față a urmărit proiectarea și implementarea unui sistem IoT inteligent pentru monitorizarea și siguranța persoanelor vârstnice. Pornind de la obiectivele formulate în subcapitolul 1.4, se poate constata că acestea au fost atinse.

A fost realizat un subsistem de achiziție a datelor de la senzori biomedicali, cu transmitere în timp real către un server centralizat (O1), și a fost dezvoltată o aplicație web completă pentru vizualizarea datelor, accesibilă atât aparținătorilor, cât și medicilor (O2). A fost definit și implementat mecanismul original de evaluare a stării de sănătate, anume health score-ul, bazat pe corelarea mai multor parametri vitali și pe praguri personalizabile (O3), și a fost integrat un sistem de notificări multi-canal declanșat automat (O4). Sistemul funcționează în regim apropiat de timp real, cu o latență redusă (O5), iar arhitectura stratificată și modulară adoptată îi asigură scalabilitatea și extensibilitatea (O6).

Principala contribuție originală a lucrării rămâne mecanismul health score, care depășește limitarea sistemelor existente bazate pe praguri unice, oferind un indice sintetic, personalizat și mai puțin predispus la alerte false. La aceasta se adaugă integrarea, într-un sistem funcțional și unitar, a unor componente care în soluțiile analizate apar doar izolat: monitorizarea continuă, alertarea activă a aparținătorului, configurarea personalizată și stocarea locală la pierderea conexiunii.

7.2 Limitări identificate

Cu toate că sistemul este funcțional, dezvoltarea sa în cadrul unei lucrări de licență a presupus o serie de limitări.

- **Hardware de prototip:** dispozitivul a fost realizat din componente de uz general, asamblate într-o formă de prototip; el nu are gradul de miniaturizare, ergonomie și rezistență mecanică al unui produs comercial.
- **Antrenarea pe date sintetice:** modelul de inteligență artificială a fost antrenat pe date generate pornind de la seturi publice, nu pe date clinice reale colectate de la pacienți, ceea ce limitează validitatea concluziilor în condiții reale.
- **Validare la scară redusă:** testarea a fost realizată în regim de laborator, pe un număr mic de dispozitive, fără un studiu de utilizare pe termen lung cu utilizatori reali.
- **Dependența de servicii externe:** funcționarea alertării prin SMS și a autentificării prin Google depinde de servicii terțe și de disponibilitatea acestora.
- **Baza de date:** soluția SQLite este potrivită pentru scara actuală, dar un număr foarte mare de utilizatori ar necesita migrarea către un sistem de baze de date de tip server.

7.3 Direcții de dezvoltare ulterioară

Sistemul dezvoltat constituie o bază solidă pentru extinderi viitoare. Printre direcțiile de dezvoltare se numără:

- antrenarea și validarea modelului de inteligență artificială pe date clinice reale, obținute în cadrul unui studiu cu pacienți, eventual cu acordul unei instituții medicale;
- dezvoltarea unei aplicații mobile native pentru aparținători, complementară interfeței web;

- integrarea de senzori suplimentari, de exemplu pentru măsurarea ECG sau a tensiunii arteriale, în scopul unei evaluări medicale mai cuprinzătoare;
- optimizarea consumului de energie al dispozitivului, pentru creșterea autonomiei bateriei;
- integrarea cu sistemele informatice medicale, pentru ca datele să poată fi consultate direct de medicul curant în fișa electronică a pacientului;
- extinderea predicției de tendințe (descrisă în subcapitolul 4.4 pentru orizontul de câteva minute) cu o analiză pe termen lung, pe zile și săptămâni, pentru detectarea degradărilor lente ale stării de sănătate.

În concluzie, LifeAlertPlus demonstrează că o soluție de monitorizare medicală accesibilă, fiabilă și inteligentă poate fi construită cu tehnologii open-source și hardware de uz general, adresând un segment real și insuficient acoperit: siguranța persoanelor vârstnice prin implicarea activă a aparținătorului.

Bibliografie

- [1] World Health Organization, *Noncommunicable diseases. Fact sheet*, 2023. <https://www.who.int/news-room/fact-sheets/detail/noncommunicable-diseases> (accesat: 29.05.2026).
- [2] Fortune Business Insights, *Internet of Medical Things (IoMT) Market Size, Share & Industry Analysis*, 2023. <https://www.fortunebusinessinsights.com/internet-of-medical-things-iomt-market-101844> (accesat: 29.05.2026).
- [3] K. Ashton, *That "Internet of Things" Thing*, RFID Journal, 1999. <https://www.rfidjournal.com/that-internet-of-things-thing> (accesat: 29.05.2026).
- [4] S. M. Lundberg, S.-I. Lee, *A Unified Approach to Interpreting Model Predictions*, în *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html (accesat: 29.05.2026).
- [5] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. <https://www.jmlr.org/papers/v12/pedregosa11a.html> (accesat: 29.05.2026).
- [6] PhysioNet, *BIDMC PPG and Respiration Dataset*, v1.0.0. <https://physionet.org/content/bidmc/1.0.0/> (accesat: 29.05.2026).
- [7] Espressif Systems, *ESP32-C3 Series Datasheet*. https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf (accesat: 29.05.2026).
- [8] Analog Devices (Maxim Integrated), *MAX30100. Pulse Oximeter and Heart-Rate Sensor IC for Wearable Health Datasheet*. <https://www.analog.com/media/en/technical-documentation/data-sheets/MAX30100.pdf> (accesat: 29.05.2026).
- [9] Analog Devices (Maxim Integrated), *DS18B20. Programmable Resolution 1-Wire Digital Thermometer Datasheet*. <https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf> (accesat: 29.05.2026).
- [10] InvenSense (TDK), *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*. <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf> (accesat: 29.05.2026).
- [11] u-blox, *NEO-6 u-blox 6 GPS Modules Data Sheet*. https://content.u-blox.com/sites/default/files/products/documents/NEO-6_DataSheet_%28GPS.G6-HW-09005%29.pdf (accesat: 29.05.2026).
- [12] Microsoft, *ASP.NET Core documentation*. <https://learn.microsoft.com/en-us/aspnet/core/> (accesat: 29.05.2026).
- [13] Microsoft, *Blazor documentation: Build client-side and server-side web apps with .NET*. <https://learn.microsoft.com/en-us/aspnet/core/blazor/> (accesat: 29.05.2026).
- [14] Docker Inc., *Docker Documentation*. <https://docs.docker.com/> (accesat: 29.05.2026).